

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

BÁO CÁO ĐỒ ÁN THẠC SĨ

(Theo định hướng ứng dụng)

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG AI DOANH NGHIỆP
HƯỚNG TÍCH HỢP ĐA HỆ THỐNG**

(Enterprise AI Platform for Cross-System Integration)

Học viên: Trần Quang Ninh

Chuyên ngành: Hệ thống Thông tin

Mã số: 8.48.01.04

Người hướng dẫn: PGS.TS. Trần Đình Quế

HÀ NỘI – 2026

BÁO CÁO ĐỒ ÁN THẠC SĨ

**NGHIÊN CỨU VÀ XÂY DỰNG NỀN TẢNG AI DOANH NGHIỆP HƯỚNG
TÍCH HỢP ĐA HỆ THỐNG (ENTERPRISE AI PLATFORM FOR
CROSS-SYSTEM INTEGRATION)**

HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

Học viên: Trần Quang Ninh

Chuyên ngành: Hệ thống Thông tin

Mã số: 8.48.01.04

BÁO CÁO ĐỒ ÁN THẠC SĨ

(Theo định hướng ứng dụng)

NGƯỜI HƯỚNG DẪN KHOA HỌC: PGS.TS. TRẦN ĐÌNH QUẾ

HÀ NỘI – 2026

MỤC LỤC

DANH	MỤC
BẢNG.....	5

DANH	MỤC
HÌNH.....	7

DANH	MỤC	CHỮ	CÁI	VIẾT
TẮT.....				9

MỞ ĐẦU.....				
.....12	1.	Lý	do	chọn
tài.....				đề
.....12				2. Tổng quan
về vấn đề nghiên cứu.....				14
.....14				2.1. Giới
thiệu lĩnh vực nghiên cứu.....				14
.....14				2.2.
Các hướng tiếp cận chính.....				14
.....15				2.3. Khoảng
trống nghiên cứu và vấn đề đặt				ra.....
.....15				3. Mục
.....16				đích nghiên
.....16				cứu.....
.....16				3.1. Mục tiêu
.....16				tổng quát.....
.....16				3.2. Mục
.....16				tiêu cụ thể.....
.....17				4. Đối
.....17				tượng và phạm vi nghiên cứu.....
.....17				4.1. Đối tượng nghiên cứu.....
.....17				4.2. Phạm vi nghiên cứu.....
.....18				5. Phương
.....18				pháp nghiên
.....18				cứu.....

NỘI DUNG.....	
.....22	

Chương 1: CƠ SỞ LÝ LUẬN VÀ TỔNG QUAN NGHIÊN	
CỨU.....	22
.....22	1.1. Tổng quan về trí tuệ nhân tạo và mô hình ngôn
.....22	ngữ lớn.....
.....22	1.1.1. Khái quát về trí tuệ nhân
.....22	tạo.....
.....23	1.1.2. Mô hình ngôn ngữ lớn
.....23	(LLM).....
.....26	1.1.3. Các mô hình LLM tiêu
.....26	biểu.....
.....29	1.1.4. Hạn chế của LLM
.....29	trong bài toán doanh nghiệp.....
.....30	1.2.
.....30	Retrieval-Augmented Generation (RAG).....
.....30	1.2.1. Khái niệm và nguyên lý hoạt động.....
.....31	1.2.2. Kiến trúc tổng quát của hệ thống RAG.....
.....31	1.2.3. Vai trò của RAG trong hệ thống doanh
.....33	33
.....34	1.2.4. Ưu điểm và hạn chế của
.....34	RAG.....
.....35	1.3. AI Agent và cơ chế phối
.....35	hợp đa tác nhân (Multi-Agent).....
.....35	1.3.1. Khái niệm AI
.....35	Agent.....
.....36	1.3.2. Kiến trúc tác
.....36	nhân và cơ chế lập kế hoạch.....
.....36	1.3.3. Multi-Agent

Orchestration.....	37
thách thức khi áp dụng Multi-Agent.....	38
Model Context Protocol (MCP).....	39
cảnh ra đời của MCP.....	39
trúc và cách thức hoạt động.....	40
MCP trong bài toán tích hợp hệ thống doanh nghiệp.....	41
tích hợp hệ thống doanh nghiệp (Enterprise System Integration).....	42
Tổng quan về ERP, CRM, DMS, HRM.....	42
Các phương pháp tích hợp truyền thống.....	44
Thách thức khi tích hợp AI vào hệ thống doanh nghiệp.....	46
Các nghiên cứu và hệ thống liên quan.....	47
1.6.1. Các nền tảng AI doanh nghiệp hiện có.....	47
1.6.2. Các nghiên cứu về AI Agent và Multi-Agent.....	49
1.6.3. Khoảng trống nghiên cứu.....	51
1.7. Kết luận chương 1.....	52

Chương 2: PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ NỀN TẢNG.....	54
2.1. Phân tích bài toán.....	54
2.1.1. Mô tả bài toán và bối cảnh doanh nghiệp.....	54
2.1.2. Yêu cầu chức năng.....	55
2.1.3. Yêu cầu phi chức năng.....	57
2.2. Kiến trúc tổng thể.....	59
2.2.1. Nguyên tắc thiết kế và triết lý kiến trúc.....	59
2.2.2. Kiến trúc 7 tầng (Layered Architecture).....	61
2.2.3. So sánh các phong cách kiến trúc (Monolith, Modular, Microservice).....	63
2.3. Thiết kế các module cốt lõi.....	65
2.3.1. AI Provider Abstraction Layer.....	65
2.3.2. AI Engine Core – RAG Engine và Hybrid Search.....	67
2.3.3. Agent Orchestration Layer.....	69
2.3.4. AI Gateway Layer.....	71
2.3.5. Integration Layer – MCP và Plugin SDK.....	72
2.4. Thiết kế dữ liệu và cơ sở tri thức.....	74
2.4.1. Mô hình dữ liệu quan hệ (PostgreSQL).....	74
2.4.2. Cơ sở dữ liệu vector (pgvector).....	76
2.4.3. Cơ sở dữ liệu đa tenant và bảo mật.....	77
2.4.4. Kho tri thức doanh nghiệp (Enterprise Knowledge Base).....	78
2.5. Thiết kế giao thức tích hợp và bảo mật.....	79
2.5.1. Giao thức MCP Server và Client.....	79
2.5.2. Cơ chế Plugin và Hot-reload.....	81
2.5.3. Xác thực, phân quyền và Audit log.....	82
2.6. Quy trình triển khai theo	

Phase.....	84	2.6.1. Tổng quan 5
Phase.....	84	2.6.2. Chiến lược
Walking Skeleton và Vertical Slice.....	85	2.6.3. Quản lý
Interface giữa các Phase.....	86	2.7. Kết luận
chương 2.....	88	

Chương 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ.....	89	3.1. Thiết lập thực nghiệm.....	89
và phương pháp đánh giá.....	89	3.1.1. Mục tiêu	
trường thực nghiệm và cấu hình hệ thống.....	90	3.1.2. Môi	
dữ liệu thực nghiệm và kịch bản kiểm thử.....	92	3.1.3. Bộ	
dựng hệ thống thực nghiệm.....	93	3.2. Xây	
Tổng quan các thành phần triển khai.....	93	3.2.1.	
Triển khai Phase 1 – Foundation (Walking Skeleton).....	95	3.2.2.	
Triển khai Phase 2 – AI Engine và RAG.....	97	3.2.3.	
Triển khai Phase 3 – Agent và Integration.....	98	3.2.4.	
Kết quả đầu ra của hệ thống và minh chứng.....	100	3.2.5.	
Kết quả và đánh giá.....	101	3.3.	
3.3.1. Đánh giá khả năng tích hợp đa hệ thống.....	101		
3.3.2. Đánh giá chất lượng truy xuất tri thức (RAGAS).....	103		
3.3.3. Đánh giá khả năng phối hợp đa tác nhân (Multi-Agent).....	105		
3.3.4. Đánh giá hiệu năng và khả năng mở rộng.....	106		
3.3.5. So sánh với các phương pháp truyền thống.....	108		
3.3.6. Tổng hợp kết quả đánh giá.....	110		
3.4. Kết luận chương 3.....	112		

KẾT LUẬN.....	113
---------------	-----

DANH MỤC CÁC TÀI LIỆU THAM KHẢO.....	118
--------------------------------------	-----

DANH MỤC BẢNG

Chương 1

Bảng 1.1. Ưu điểm và hạn chế của RAG.....	34	Bảng 1.2. So sánh một số nền tảng AI doanh nghiệp hiện có.....	48
Bảng 1.3. Một số nghiên cứu ứng dụng AI Agent trong doanh nghiệp.....	50		

Chương 2

Bảng 2.1. Yêu cầu chức năng của hệ thống.....	56
Bảng 2.2. Yêu cầu phi chức năng của hệ thống.....	58
Bảng 2.3. So sánh các phong cách kiến trúc.....	63
Bảng 2.4. Mô tả chi tiết 7 Layer trong kiến trúc.....	64
Bảng 2.5. Bảng so sánh các communication pattern.....	72
Bảng 2.6. Các module chính của Platform Core.....	83
Bảng 2.7. Phân bố module theo 5 Phase triển khai.....	84
Bảng 2.8. Interfaces được chốt qua từng Phase.....	87

Chương 3

Bảng 3.1. Mục tiêu và phương pháp đánh giá thực nghiệm.....	89
Bảng 3.2. Môi trường phần cứng và phần mềm.....	91
Bảng 3.3. Cấu hình LLM và Embedding sử dụng trong thực nghiệm.....	91
Bảng 3.4. Thống kê bộ dữ liệu thực nghiệm.....	92
Bảng 3.5. Kịch bản thực nghiệm.....	93
Bảng 3.6. Thành phần triển khai trong hệ thống thực nghiệm.....	94
Bảng 3.7. Kết quả triển khai các module theo Phase.....	95
Bảng 3.8. Kết quả đánh giá khả năng tích hợp đa hệ thống.....	101
Bảng 3.9. Kết quả đánh giá chất lượng RAGAS.....	103
Bảng 3.10. Kết quả đánh giá Multi-Agent.....	105
Bảng 3.11. Kết quả đánh giá hiệu năng hệ thống.....	106
Bảng 3.12. So sánh chi phí giữa các cấu hình LLM.....	108
Bảng 3.13. Tổng hợp kết quả theo các kịch bản thực nghiệm.....	110

DANH MỤC HÌNH

Chương 1

Hình 1.1. Lịch sử phát triển của Trí tuệ nhân tạo.....	22
Hình 1.2. Kiến trúc Transformer – nền tảng của các LLM hiện đại.....	24
Hình 1.3. So sánh các mô hình LLM phổ biến về quy mô và khả năng.....	28
Hình 1.4. Kiến trúc tổng quát của hệ thống RAG.....	31
Hình 1.5. Kiến trúc RAG nâng cao với Hybrid Search và Reranking.....	32
Hình 1.6. Vòng lặp ReAct của AI Agent.....	36
Hình 1.7. Kiến trúc Multi-Agent với Supervisor Pattern.....	38
Hình 1.8. Kiến trúc Model Context Protocol (MCP).....	40
Hình 1.9. Mô hình tích hợp AI – ERP/CRM/DMS qua MCP.....	41
Hình 1.10. Tổng quan các hệ thống doanh nghiệp và thách thức tích hợp.....	45

Chương 2

Hình 2.1. Tám nguyên tắc thiết kế của Enterprise AI Platform.....60

Hình 2.2. So sánh triết lý Monolith – Modular Monolith – Microservice.....63

Hình 2.3. Kiến trúc 7 Layer tổng quan của nền tảng.....61

Hình 2.4. Kiến trúc chi tiết AI Provider Abstraction Layer.....66

Hình 2.5. Kiến trúc AI Engine Core (RAG Engine và Hybrid Search).....68

Hình 2.6. Kiến trúc Agent Orchestration Layer.....70

Hình 2.7. Kiến trúc AI Gateway với các middleware bảo mật.....71

Hình 2.8. Kiến trúc Integration Layer với MCP và Plugin SDK.....73

Hình 2.9. Sơ đồ ERD cơ sở dữ liệu PostgreSQL.....75

Hình 2.10. Cấu trúc bảng vector embedding trong pgvector.....76

Hình 2.11. Cơ chế Multi-tenant với Row-Level Security (RLS).....77

Hình 2.12. Kiến trúc Enterprise Knowledge Base.....78

Hình 2.13. Mô hình triển khai MCP Server và Client trong doanh nghiệp.....80

Hình 2.14. Cơ chế Plugin với Hot-reload.....82

Hình 2.15. Lộ trình triển khai 5 Phase.....85

Hình 2.16. Nguyên tắc Walking Skeleton và Vertical Slice.....86

Chương 3

Hình 3.1. Giao diện chính của hệ thống thực nghiệm.....96

Hình 3.2. Giao diện chat với Ollama local (Phase 1).....96

Hình 3.3. Giao diện upload tài liệu và hỏi đáp RAG (Phase 2).....98

Hình 3.4. Giao diện Agent với tool calling (Phase 3).....99

Hình 3.5. Minh chứng tích hợp ERP/CRM/DMS qua MCP.....100

Hình 3.6. Biểu đồ so sánh kết quả RAGAS theo provider.....104

Hình 3.7. Biểu đồ latency theo từng phase của pipeline.....107

Hình 3.8. Biểu đồ so sánh chi phí giữa các LLM provider.....108

Hình 3.9. Biểu đồ tăng trưởng throughput khi scale.....109

DANH MỤC CHỮ CÁI VIẾT TẮT

Từ viết tắt	Tiếng Anh	Nghĩa tiếng Việt
AI	Artificial Intelligence	Trí tuệ nhân tạo
ABAC	Attribute-Based Access Control	Kiểm soát truy cập dựa trên thuộc tính
API	Application Programming Interface	Giao diện lập trình ứng dụng
BPM	Business Process Management	Quản lý quy trình nghiệp vụ

BERT	Bidirectional Encoder Representations from Transformers	Mô hình biểu diễn mã hóa hai chiều dựa trên Transformer
BM25	Best Matching 25	Thuật toán xếp hạng tìm kiếm từ khóa
BPMN	Business Process Model and Notation	Chuẩn mô hình hóa quy trình nghiệp vụ
CI/CD	Continuous Integration / Continuous Deployment	Tích hợp liên tục / Triển khai liên tục
CPU	Central Processing Unit	Bộ xử lý trung tâm
CRM	Customer Relationship Management	Quản lý quan hệ khách hàng
DDD	Domain-Driven Design	Thiết kế hướng miền
DI	Dependency Injection	Tiêm phụ thuộc
DMS	Document Management System	Hệ thống quản lý tài liệu
ESB	Enterprise Service Bus	Bus dịch vụ doanh nghiệp
ERP	Enterprise Resource Planning	Hoạch định nguồn lực doanh nghiệp
GDPR	General Data Protection Regulation	Quy định bảo vệ dữ liệu chung (EU)
GPT	Generative Pre-trained Transformer	Mô hình Transformer tiền huấn luyện sinh văn bản
GPU	Graphics Processing Unit	Bộ xử lý đồ họa
gRPC	gRPC Remote Procedure Call	Giao thức gọi hàm từ xa hiệu năng cao
HITL	Human-in-the-Loop	Con người trong vòng lặp
HRM	Human Resource Management	Quản lý nguồn nhân lực
HTTP	HyperText Transfer Protocol	Giao thức truyền tải siêu văn bản
HTTPS	HyperText Transfer Protocol Secure	HTTP có mã hóa
JWT	JSON Web Token	Token xác thực dạng JSON
K8s	Kubernetes	Hệ thống điều phối container
LLM	Large Language Model	Mô hình ngôn ngữ lớn
LSP	Language Server Protocol	Giao thức server ngôn ngữ
MCP	Model Context Protocol	Giao thức ngữ cảnh mô hình
MoE	Mixture of Experts	Hỗn hợp chuyên gia
NLG	Natural Language Generation	Sinh ngôn ngữ tự nhiên
NLP	Natural Language Processing	Xử lý ngôn ngữ tự nhiên

NLU	Natural Language Understanding	Hiểu ngôn ngữ tự nhiên
OLLM	Open-source Large Language Model	Mô hình ngôn ngữ lớn mã nguồn mở
ORM	Object-Relational Mapping	Ảnh xạ đối tượng - quan hệ
POC	Proof of Concept	Bằng chứng khái niệm
RBAC	Role-Based Access Control	Kiểm soát truy cập dựa trên vai trò
RAG	Retrieval-Augmented Generation	Sinh nội dung tăng cường bằng truy xuất
RAGAS	Retrieval-Augmented Generation Assessment	Khung đánh giá hệ thống RAG
RAM	Random Access Memory	Bộ nhớ truy cập ngẫu nhiên
ReAct	Reason + Act	Lý luận + Hành động
REST	Representational State Transfer	Kiến trúc truyền trạng thái đại diện
RLHF	Reinforcement Learning from Human Feedback	Học tăng cường từ phản hồi con người
RLS	Row-Level Security	Bảo mật cấp hàng (PostgreSQL)
RRF	Reciprocal Rank Fusion	Hợp nhất xếp hạng nghịch đảo
SDK	Software Development Kit	Bộ công cụ phát triển phần mềm
SLA	Service Level Agreement	Thỏa thuận cấp độ dịch vụ
SQL	Structured Query Language	Ngôn ngữ truy vấn có cấu trúc
SSE	Server-Sent Events	Sự kiện gửi từ máy chủ
SSO	Single Sign-On	Đăng nhập một lần
STT	Speech-to-Text	Chuyển giọng nói thành văn bản
TF-IDF	Term Frequency – Inverse Document Frequency	Tần suất thuật ngữ – tần suất nghịch tài liệu
TPU	Tensor Processing Unit	Bộ xử lý tensor
TTS	Text-to-Speech	Chuyển văn bản thành giọng nói
UI	User Interface	Giao diện người dùng
URI	Uniform Resource Identifier	Định danh tài nguyên thống nhất
UUID	Universally Unique Identifier	Định danh duy nhất toàn cục
VDB	Vector Database	Cơ sở dữ liệu vector
WAF	Web Application Firewall	Tường lửa ứng dụng web
YAML	YAML Ain't Markup Language	Định dạng dữ liệu YAML

MỞ ĐẦU

1. Lý do chọn đề tài

Trong những năm gần đây, sự bùng nổ của trí tuệ nhân tạo (Artificial Intelligence - AI), đặc biệt là các mô hình ngôn ngữ lớn (Large Language Models - LLM), đã tạo ra một cuộc cách mạng trong cách thức con người tương tác với hệ thống thông tin. Các mô hình như GPT-4, Claude, Gemini hay các mô hình mã nguồn mở như Llama, Mistral, DeepSeek không chỉ dừng lại ở khả năng sinh văn bản mà còn có thể lập luận, phân tích, lập kế hoạch và gọi các công cụ (function calling) để thực hiện tác vụ phức tạp. Điều này mở ra cơ hội ứng dụng AI vào hầu hết các ngành nghề, từ y tế, giáo dục, tài chính đến đặc biệt là trong lĩnh vực doanh nghiệp.

Tuy nhiên, việc ứng dụng AI vào các hệ thống doanh nghiệp hiện hữu (ERP, CRM, DMS, HRM, Workflow...) vẫn còn gặp nhiều rào cản. Theo khảo sát của các tổ chức nghiên cứu, phần lớn các doanh nghiệp khi muốn tích hợp AI vào hệ thống thường phải đối mặt với bốn vấn đề lớn: chi phí tích hợp cao, thời gian triển khai lâu, yêu cầu chuyên môn sâu về cả AI lẫn hệ thống và khó mở rộng khi muốn áp dụng cho nhiều hệ thống khác nhau. Một doanh nghiệp có hệ thống ERP muốn bổ sung chatbot hỗ trợ tra cứu thông tin phải xây dựng từ đầu; khi muốn tích hợp thêm với CRM lại phải làm lại; các giải pháp AI cho từng hệ thống thường không liên thông với nhau, dẫn đến dữ liệu và tri thức bị phân mảnh.

Bên cạnh đó, một thách thức cốt lõi khác là sự phụ thuộc vào AI provider. Mỗi nhà cung cấp (OpenAI, Anthropic, Google, các mô hình mã nguồn mở...) có API riêng, cách tính phí riêng, giới hạn ngữ cảnh khác nhau và hành vi khác nhau. Việc chuyển đổi giữa các provider, hoặc sử dụng đồng thời nhiều provider, đòi hỏi phải thay đổi mã nguồn tích hợp. Điều này khiến doanh nghiệp bị khóa vào một nhà cung cấp cụ thể (vendor lock-in), đặc biệt nguy hiểm khi giá cả hoặc chính sách thay đổi.

Đồng thời, sự ra đời của giao thức Model Context Protocol (MCP) vào năm 2024 bởi Anthropic đã chuẩn hóa cách thức AI Agent tương tác với các hệ thống và công cụ bên ngoài, mở ra một hướng tiếp cận mới: AI có thể trở thành "cầu nối thông minh" giữa người dùng và hệ thống mà không cần phải sửa đổi hệ thống gốc.

Xuất phát từ những vấn đề trên, việc nghiên cứu và xây dựng một nền tảng AI doanh nghiệp (Enterprise AI Platform) hướng tới tích hợp đa hệ thống là cần thiết và có ý nghĩa cả về mặt khoa học lẫn thực tiễn. Mục tiêu là tạo ra một sứ mệnh rõ ràng: "Chỉ cần cắm AI vào hệ thống khác là có thể sử dụng được" — nghĩa là doanh nghiệp có thể tích hợp AI vào ERP, CRM, DMS, HRM chỉ trong thời gian ngắn nhất, chi phí thấp nhất, độc lập với AI provider cụ thể và có khả năng mở rộng cao.

Đề tài không chỉ đóng góp vào việc ứng dụng các công nghệ AI hiện đại (LLM, RAG, AI Agent, MCP) vào thực tế doanh nghiệp mà còn xây dựng một mô

hình kiến trúc có thể tham chiếu cho các dự án AI doanh nghiệp tại Việt Nam và khu vực.

2. Tổng quan về vấn đề nghiên cứu

2.1. Giới thiệu lĩnh vực nghiên cứu

Tích hợp AI vào hệ thống doanh nghiệp là một lĩnh vực nghiên cứu đang phát triển rất nhanh, nằm ở giao điểm của nhiều ngành: trí tuệ nhân tạo, hệ thống phân tán, tích hợp hệ thống, kiến trúc phần mềm và quản trị doanh nghiệp. Lĩnh vực này có thể được phân thành các hướng nghiên cứu chính:

- Hệ thống gợi ý và trợ lý thông minh (Intelligent Assistant): sử dụng LLM kết hợp RAG để hỗ trợ tra cứu thông tin, hỏi đáp và đưa ra gợi ý dựa trên dữ liệu nội bộ doanh nghiệp.
- AI Agent tự động hóa quy trình (Process Automation): sử dụng AI Agent có khả năng lập kế hoạch và gọi tool để tự động hóa các quy trình nghiệp vụ như phê duyệt đơn, tạo báo cáo, đồng bộ dữ liệu giữa các hệ thống.
- Nền tảng AI đa thuê (Multi-tenant AI Platform): kiến trúc cho phép nhiều doanh nghiệp cùng sử dụng chung một nền tảng với cấu hình và bảo mật riêng biệt.
- Chuẩn giao tiếp AI – Hệ thống: như MCP, Agent-to-Agent Protocol, các chuẩn mở nhằm chuẩn hóa cách AI tương tác với hệ thống ngoài.

2.2. Các hướng tiếp cận chính

Trong nghiên cứu và triển khai thực tế hiện nay, có ba hướng tiếp cận chính cho bài toán tích hợp AI vào doanh nghiệp:

Hướng 1: Tích hợp trực tiếp vào hệ thống hiện hữu (Embedded AI) Mỗi hệ thống (ERP, CRM...) tự tích hợp AI bằng cách gọi trực tiếp API của LLM provider. Ưu điểm là đơn giản cho từng hệ thống riêng lẻ, nhưng nhược điểm là chi phí nhân bản, khó chia sẻ tri thức giữa các hệ thống và dễ bị khóa vào provider.

Hướng 2: Xây dựng nền tảng trung gian (Middleware / Integration Platform) Một nền tảng trung gian đặt giữa các hệ thống doanh nghiệp và các AI provider, cung cấp các khả năng như: chuẩn hóa giao tiếp (provider abstraction), truy xuất tri thức chung (RAG), điều phối nhiều agent (multi-agent orchestration). Đây là hướng tiếp cận của đề tài.

Hướng 3: Sử dụng nền tảng AI thương mại SaaS Sử dụng các dịch vụ như Microsoft Copilot, Google Workspace AI, Salesforce Einstein. Ưu điểm là triển khai nhanh nhưng chi phí cao, khó tùy biến, phụ thuộc hoàn toàn vào nhà cung cấp và không phù hợp với các hệ thống tự xây dựng.

2.3. Khoảng trống nghiên cứu và vấn đề đặt ra

Mặc dù cả ba hướng trên đều có nghiên cứu và sản phẩm thương mại, vẫn còn những khoảng trống quan trọng:

- Thiếu một kiến trúc tổng thể kết hợp đồng thời LLM, RAG, AI Agent, MCP trong cùng một nền tảng có khả năng tích hợp với nhiều hệ thống doanh nghiệp khác nhau mà không phải viết lại từ đầu cho mỗi hệ thống.
- Thiếu cơ chế "Plug and Play" thực sự: hầu hết các giải pháp hiện nay vẫn yêu cầu can thiệp sâu vào hệ thống đích hoặc cấu hình phức tạp. Chưa có giải pháp nào đạt được mức "cắm vào là chạy" (zero config) ở mức tổng quát.
- Vendor lock-in chưa được giải quyết triệt để: các giải pháp thường gắn liền với một provider cụ thể hoặc một bộ công nghệ cụ thể. Khi muốn chuyển đổi, doanh nghiệp thường phải làm lại.
- Thiếu chiến lược triển khai tiết kiệm tài nguyên AI trong khi vẫn phải đáp ứng yêu cầu doanh nghiệp: nhiều công trình xây dựng kiến trúc lý thuyết đầy đủ nhưng thiếu hướng dẫn triển khai từng bước (Phase) để có thể áp dụng vào đội ngũ từ 20-100 lập trình viên.
- Ứng dụng cho bối cảnh doanh nghiệp Việt Nam: các nghiên cứu hiện có chủ yếu ở thị trường quốc tế. Cần một kiến trúc tham chiếu phù hợp với đặc thù hệ thống doanh nghiệp Việt Nam (quy mô vừa và nhỏ, hạ tầng tự xây, yêu cầu chạy cả offline lẫn hybrid).

Do đó, đề tài đặt ra vấn đề nghiên cứu: Làm thế nào để xây dựng một nền tảng AI doanh nghiệp có khả năng tích hợp nhanh chóng vào nhiều hệ thống khác nhau, độc lập với AI provider, có khả năng mở rộng và đạt chuẩn doanh nghiệp về bảo mật, kiểm toán?

3. Mục đích nghiên cứu

3.1. Mục tiêu tổng quát

Mục tiêu tổng quát của đề tài là nghiên cứu và xây dựng một nền tảng AI doanh nghiệp (Enterprise AI Platform) hướng tới tích hợp đa hệ thống (multi-system integration), cho phép doanh nghiệp "cắm AI vào" các hệ thống ERP, CRM, DMS, HRM, Workflow... một cách nhanh chóng, chi phí thấp, độc lập với AI provider cụ thể và có khả năng mở rộng cao. Nền tảng được thiết kế theo triết lý "Plug & Play — Provider Agnostic — Domain Aware — Enterprise Grade", kết hợp đồng thời các công nghệ hiện đại gồm LLM, RAG, AI Agent, Multi-Agent Orchestration và giao thức MCP.

3.2. Mục tiêu cụ thể

Để đạt được mục tiêu tổng quát, đề tài đặt ra các mục tiêu cụ thể sau:

1. Nghiên cứu cơ sở lý thuyết: tổng hợp và phân tích các công nghệ nền tảng gồm LLM, RAG, Hybrid Search, Reranking, AI Agent, Multi-Agent Orchestration, giao thức MCP và các phương pháp tích hợp hệ thống doanh nghiệp, từ đó xác định khoảng trống nghiên cứu và xu hướng phát triển.
2. Khảo sát thực tiễn: nghiên cứu các nền tảng AI doanh nghiệp hiện có (cả thương mại và nghiên cứu), phân tích ưu nhược điểm và xác định yêu cầu thực tế khi áp dụng tại Việt Nam.
3. Đề xuất kiến trúc nền tảng: xây dựng một kiến trúc phân lớp (7-layer) gồm Presentation, AI Gateway, Agent Orchestration, AI Engine Core, Provider Abstraction, Integration và Infrastructure; trong đó tích hợp các công nghệ AI mới nhất vào một thiết kế thống nhất, có khả năng triển khai theo từng giai đoạn (phase).
4. Phát triển hệ thống thực nghiệm: triển khai nền tảng ở mức chạy được (working software) với các module cốt lõi: AI Provider Abstraction (hỗ trợ nhiều LLM), RAG Engine với Hybrid Search và Reranking, Agent Orchestration với Multi-Agent, MCP Server để tích hợp với hệ thống ngoài, cơ chế Plugin và bảo mật đa tenant.
5. Đánh giá hiệu quả: thực hiện các kịch bản đánh giá về (i) khả năng tích hợp đa hệ thống; (ii) chất lượng truy xuất tri thức (RAGAS: Context Relevance, Faithfulness, Answer Relevancy); (iii) khả năng phối hợp đa tác nhân; (iv) hiệu năng và khả năng mở rộng; (v) so sánh với các phương pháp truyền thống.

4. Đối tượng và phạm vi nghiên cứu

4.1. Đối tượng nghiên cứu

Đối tượng nghiên cứu của luận văn bao gồm:

- Các mô hình ngôn ngữ lớn (LLM) và đặc tính kỹ thuật phục vụ cho việc triển khai trong môi trường doanh nghiệp (multi-provider, latency, cost, safety).
- Kỹ thuật Retrieval-Augmented Generation (RAG), Hybrid Search, Reranking, Vector Database và các phương pháp nâng cao chất lượng truy xuất.
- Kiến trúc AI Agent, Multi-Agent Orchestration và các cơ chế lập kế hoạch – thực thi tác vụ (task planning, tool calling, workflow).
- Giao thức Model Context Protocol (MCP) và các chuẩn giao tiếp AI – hệ thống khác.
- Kiến trúc tích hợp hệ thống doanh nghiệp (Enterprise Integration Architecture), bao gồm: ERP, CRM, DMS, HRM, Workflow, BI, Database.
- Các yêu cầu doanh nghiệp về bảo mật (multi-tenancy, RBAC, audit log, secrets management) và khả năng mở rộng.

4.2. Phạm vi nghiên cứu

Luận văn tập trung nghiên cứu và xây dựng một nền tảng kiến trúc tham chiếu (reference architecture) cho AI doanh nghiệp, có khả năng tích hợp với đa dạng hệ thống hiện hữu. Phạm vi cụ thể:

- Về công nghệ: tập trung vào tích hợp LLM (đa provider), RAG, AI Agent/Multi-Agent, MCP, plugin SDK và các pattern kiến trúc phần mềm kèm theo. Không đi sâu vào huấn luyện mô hình từ đầu (pre-training, fine-tuning ở quy mô lớn).
- Về lĩnh vực: hệ thống doanh nghiệp (enterprise systems), bao gồm nhưng không giới hạn ở ERP, CRM, DMS, HRM, Workflow, BI. Có thể mở rộng sang các hệ thống chuyên ngành trong tương lai.
- Về quy mô: nền tảng được thiết kế cho quy mô từ doanh nghiệp nhỏ và vừa đến doanh nghiệp lớn; chạy được cả môi trường on-premise, private cloud lẫn hybrid cloud. Triển khai đa tenant từ 20 đến hàng ngàn người dùng trong một tenant.
- Về mặt ngôn ngữ và bối cảnh: hỗ trợ tiếng Việt là ngôn ngữ chính và tiếng Anh là ngôn ngữ bổ trợ, phù hợp với bối cảnh doanh nghiệp Việt Nam và khả năng mở rộng khu vực.
- Về dữ liệu: sử dụng dữ liệu mô phỏng và dữ liệu công khai phục vụ thực nghiệm, đánh giá. Các dữ liệu nội bộ riêng của từng doanh nghiệp chỉ được tích hợp khi có sự đồng ý và tuân thủ chính sách bảo mật.
- Về thời gian: nghiên cứu, xây dựng và đánh giá được thực hiện trong thời gian triển khai luận văn. Các công nghệ AI có tốc độ phát triển nhanh, do đó một số chi tiết kỹ thuật có thể thay đổi theo thời gian nhưng kiến trúc tổng thể vẫn giữ tính tham chiếu.

Ngoài phạm vi: nghiên cứu không đi sâu vào các bài toán ngoài doanh nghiệp (ví dụ: chatbot giải trí, sáng tạo nội dung thuần túy, game), cũng không xây dựng một LLM mới từ đầu. Nền tảng giả định sử dụng các LLM có sẵn (cả thương mại và mã nguồn mở).

5. Phương pháp nghiên cứu

Luận văn được thực hiện theo phương pháp nghiên cứu kết hợp giữa nghiên cứu lý thuyết và nghiên cứu thực nghiệm, nhằm đảm bảo vừa có cơ sở khoa học chặt chẽ vừa có tính ứng dụng thực tiễn cao.

5.1 Nghiên cứu cơ sở lý thuyết

- Thu thập, tổng hợp và phân tích các tài liệu khoa học, bài báo, sách, tài liệu kỹ thuật về LLM, RAG, AI Agent, MCP, kiến trúc tích hợp hệ thống doanh nghiệp và các pattern kiến trúc phần mềm hiện đại.

- Khảo sát các công trình nghiên cứu trong và ngoài nước nhằm xác định các hướng tiếp cận hiện có, những hạn chế và khoảng trống nghiên cứu cần giải quyết.
- Phân tích các tiêu chuẩn đánh giá hệ thống RAG (RAGAS), tiêu chuẩn đánh giá agent và các khung chất lượng phần mềm phù hợp.

5.2 Thu thập và xây dựng dữ liệu

- Thu thập tài liệu kỹ thuật, hướng dẫn sử dụng, đặc tả API của các LLM provider (OpenAI, Anthropic, Google, Ollama, Azure OpenAI...) phục vụ cho việc xây dựng lớp provider abstraction.
- Xây dựng bộ dữ liệu mô phỏng cho kho tri thức doanh nghiệp, bao gồm: tài liệu nội bộ mẫu, danh sách nhân viên, danh mục sản phẩm, quy trình nghiệp vụ và các tình huống hỏi đáp.
- Chuẩn hóa và làm sạch dữ liệu, tổ chức dưới dạng cơ sở tri thức có cấu trúc và phi cấu trúc phục vụ truy xuất.

5.3 Xây dựng hệ thống truy xuất tri thức

- Chuyển đổi dữ liệu văn bản thành vector embedding thông qua các mô hình embedding (Gemini Embedding, OpenAI Embedding, sentence-transformer...).
- Lưu trữ và truy xuất trên cơ sở dữ liệu vector (pgvector, Qdrant, Milvus hoặc các lựa chọn tương đương).
- Thiết kế cơ chế Hybrid Search kết hợp tìm kiếm ngữ nghĩa (dense) và tìm kiếm từ khóa (sparse), bổ sung Cross-Encoder Reranking và nén ngữ cảnh nhằm nâng cao chất lượng truy xuất.

5.4 Xây dựng mô hình thực nghiệm

- Phát triển nền tảng theo kiến trúc phân lớp đề xuất, triển khai các module cốt lõi: Provider Abstraction, AI Engine Core, Agent Orchestration, AI Gateway, Integration (MCP, Plugin) và Platform Core.
- Tích hợp nhiều LLM provider thông qua abstraction layer để chứng minh tính độc lập với nhà cung cấp.
- Tích hợp AI Agent với khả năng lập kế hoạch và gọi tool, thử nghiệm với các kịch bản nghiệp vụ thực tế (tạo đơn nghỉ phép, tra cứu khách hàng, tạo báo cáo...).
- Triển khai MCP Server cho phép hệ thống ngoài kết nối và tương tác với nền tảng.
- Triển khai môi trường thực nghiệm với Docker Compose, CI/CD cơ bản, đảm bảo reproducibility.

5.5 Đánh giá hệ thống

Đề tài sử dụng phương pháp đánh giá kết hợp định lượng và định tính:

Đánh giá định lượng: - Đo lường chất lượng truy xuất tri thức thông qua bộ chỉ số RAGAS gồm: Context Relevance, Context Recall, Faithfulness, Answer Relevancy. - Đo thời gian phản hồi (latency), thông lượng (throughput) của từng thành phần trong pipeline. - Đo chi phí sử dụng token và hiệu quả tài nguyên trên các cấu hình khác nhau.

Đánh giá định tính: - Đánh giá khả năng tích hợp đa hệ thống qua các kịch bản mô phỏng (một agent thao tác đồng thời trên ERP, CRM và DMS). - Đánh giá khả năng phối hợp đa tác nhân (multi-agent) qua các tình huống phức tạp đòi hỏi nhiều bước và nhiều nguồn tri thức. - So sánh kết quả giữa cấu hình cá nhân hóa (dựa trên tenant, role, ngữ cảnh) và cấu hình mặc định.

5.6 Công cụ và môi trường nghiên cứu

- Ngôn ngữ lập trình: .NET 8 (C#) cho backend, TypeScript/Next.js cho frontend, Python cho các script tiện ích.
- Mô hình ngôn ngữ lớn: đa dạng gồm OpenAI GPT-4o, Anthropic Claude 3.5, Google Gemini 2.5, Ollama local (Llama 3.2, Mistral, DeepSeek).
- Mô hình embedding: Gemini Embedding, OpenAI text-embedding-3 và một số mô hình mã nguồn mở (BGE, mE5).
- Cơ sở dữ liệu: PostgreSQL 16 + pgvector, Redis (cache), Docker cho container hóa.
- Công cụ khác: Serilog (logging), OpenTelemetry (observability), Docker Compose, GitHub Actions (CI/CD cơ bản), Swagger/OpenAPI (mô tả API).
- Môi trường phát triển: Visual Studio Code, .NET CLI, Node.js, các extension hỗ trợ.

Chương 1: CƠ SỞ LÝ LUẬN VÀ TỔNG QUAN NGHIÊN CỨU

Chương này trình bày nền tảng lý thuyết và tổng quan nghiên cứu liên quan đến đề tài, bao gồm: (i) trí tuệ nhân tạo và mô hình ngôn ngữ lớn; (ii) kỹ thuật Retrieval-Augmented Generation; (iii) AI Agent và cơ chế phối hợp đa tác nhân; (iv) giao thức Model Context Protocol; (v) tích hợp hệ thống doanh nghiệp; (vi) khảo sát các nghiên cứu và hệ thống liên quan.

1.1. Tổng quan về trí tuệ nhân tạo và mô hình ngôn ngữ lớn

1.1.1. Khái quát về trí tuệ nhân tạo

Trí tuệ nhân tạo (Artificial Intelligence - AI) là lĩnh vực nghiên cứu và phát triển các hệ thống máy tính có khả năng thực hiện những tác vụ mà khi con người thực hiện đòi hỏi trí thông minh [1]. Lịch sử phát triển của AI trải qua nhiều giai đoạn với các trường phái khác nhau: AI biểu tượng (symbolic AI) trong những năm 1950-1980, AI kết nối (connectionism) với mạng nơ-ron nhân tạo, AI thống kê (statistical AI) từ những năm 1990, và gần đây là AI học sâu (deep learning) kết hợp với dữ liệu lớn.

Trong thập niên 2010-2020, AI đạt được những bước tiến đột phá nhờ sự kết hợp của ba yếu tố: (i) sức mạnh tính toán tăng theo cấp số nhân, đặc biệt với GPU và TPU chuyên dụng; (ii) khối lượng dữ liệu khổng lồ được tạo ra từ Internet, mạng xã hội và các thiết bị cảm biến; (iii) các thuật toán học sâu ngày càng tinh vi, đặc biệt là kiến trúc Transformer [2]. Sự kết hợp này đã tạo ra những mô hình có khả năng vượt trội so với con người trong nhiều tác vụ cụ thể: nhận diện hình ảnh, dịch máy, chơi cờ vây, chẩn đoán y tế...

Từ năm 2020, một hướng phát triển mới nổi lên mang tính cách mạng là các mô hình ngôn ngữ lớn (Large Language Models - LLM). Các mô hình này thể hiện khả năng tổng quát hóa đáng kinh ngạc trên nhiều tác vụ ngôn ngữ, từ viết văn, lập trình, phân tích dữ liệu đến lý luận logic, với chất lượng gần với con người. Sự ra đời của ChatGPT vào cuối năm 2022 đã đánh dấu một bước ngoặt lớn trong việc đưa AI đến với người dùng phổ thông và doanh nghiệp.

Tuy nhiên, các mô hình LLM cũng đặt ra nhiều thách thức mới về chi phí tính toán, an toàn dữ liệu, kiểm soát đầu ra và vấn đề "ảo giác" (hallucination). Do đó, việc nghiên cứu các kiến trúc cho phép khai thác sức mạnh của LLM một cách an toàn, hiệu quả và phù hợp với từng ngữ cảnh ứng dụng là một trong những hướng nghiên cứu quan trọng nhất hiện nay.

1.1.2. Mô hình ngôn ngữ lớn (LLM)

a. Khái niệm

Mô hình ngôn ngữ lớn (Large Language Model - LLM) là nhóm mô hình học sâu được huấn luyện trên khối lượng rất lớn dữ liệu văn bản, có khả năng hiểu và sinh ngôn ngữ tự nhiên ở mức độ gần với con người [3]. Hầu hết các LLM hiện đại đều dựa trên kiến trúc Transformer [2], cho phép mô hình đánh giá mức độ liên quan giữa các token trong toàn bộ chuỗi đầu vào thông qua cơ chế attention, thay vì xử lý tuần tự như các kiến trúc RNN hay LSTM trước đó. Điều này giúp cải thiện khả năng nắm bắt ngữ cảnh dài và tăng hiệu quả tính toán.

Một LLM thường có số lượng tham số (parameters) rất lớn, từ vài tỷ (VinaLlama 7B, Mistral 7B) đến hàng trăm tỷ tham số (GPT-4 ước tính khoảng 1.8 nghìn tỷ, Claude 3 Opus khoảng 2 nghìn tỷ, Llama 3.1 405B). Quy mô tham số này cho phép mô hình "lưu trữ" một lượng lớn tri thức về ngôn ngữ, sự kiện và kỹ năng lý luận.

Quá trình phát triển một LLM thường qua hai giai đoạn chính: - Pretraining: mô hình học các đặc trưng ngôn ngữ tổng quát từ khối lượng lớn dữ liệu (thường là dữ liệu không gán nhãn), sử dụng các tác vụ như dự đoán token tiếp theo (next token prediction) hoặc masked language modeling. Giai đoạn này đòi hỏi tài nguyên tính toán rất lớn (hàng nghìn GPU trong nhiều tuần). - Fine-tuning: sau khi pretraining, mô hình có thể được tinh chỉnh cho các tác vụ cụ thể (hỏi đáp, tóm tắt, phân loại, sinh nội dung) hoặc được căn chỉnh theo ý muốn con người thông qua RLHF (Reinforcement Learning from Human Feedback).

Ngoài ra, kỹ thuật prompt engineering cho phép sử dụng một LLM đã huấn luyện sẵn để thực hiện nhiều tác vụ khác nhau chỉ bằng cách thay đổi câu lệnh đầu vào, không cần fine-tuning. Đây là cách tiếp cận được đề tài sử dụng để giữ chi phí tích hợp thấp.

b. Các mô hình LLM tiêu biểu

GPT là họ mô hình của OpenAI dựa trên phần Decoder của Transformer, sinh văn bản theo cơ chế tự hồi quy bằng cách dự đoán token tiếp theo dựa trên các token đã xuất hiện trước đó [2], [4]. Các phiên bản GPT quy mô lớn (GPT-3.5, GPT-4, GPT-4o, GPT-4o mini) cho thấy khả năng thực hiện nhiều nhiệm vụ xử lý ngôn ngữ tự nhiên thông qua cách thiết kế đầu vào (prompt) phù hợp, đặc biệt trong bối cảnh few-shot learning và zero-shot learning.

Một số đặc điểm chính của GPT: - Được huấn luyện trên khối lượng lớn dữ liệu văn bản đa ngôn ngữ. - Có khả năng sinh văn bản mạch lạc theo ngữ cảnh dài. - Hỗ trợ tool calling (function calling) cho phép kết nối với hệ thống ngoài. - Cửa sổ ngữ cảnh (context window) ngày càng được mở rộng: từ 4K (GPT-3.5), 8K, 16K, 32K, 128K (GPT-4 Turbo) đến 1M token (GPT-4.1).

Claude là họ mô hình do Anthropic phát triển, nổi bật với khả năng xử lý ngữ cảnh dài (200K token trở lên), cửa sổ ngữ cảnh rất lớn (Claude 3.5 Sonnet hỗ trợ 200K, Claude 4 hỗ trợ đến 1M), và có năng lực lý luận (reasoning) tốt. Đặc biệt, Anthropic là đơn vị khởi xướng giao thức Model Context Protocol (MCP), đây là chuẩn mở cho phép kết nối AI với các công cụ và nguồn dữ liệu. Claude đặc biệt phù hợp cho các tác vụ đòi hỏi phân tích sâu và tuân thủ an toàn.

Gemini là mô hình ngôn ngữ lớn đa phương thức do Google DeepMind phát triển, có khả năng xử lý nhiều dạng dữ liệu (văn bản, hình ảnh, âm thanh, video) trong cùng một kiến trúc [5]. Các phiên bản Gemini 1.5 và 2.5 có cửa sổ ngữ cảnh rất lớn (1M đến 2M token), phù hợp cho các tác vụ cần phân tích nhiều tài liệu. Gemini cũng cung cấp mô hình embedding chất lượng cao (Gemini Embedding) tích hợp tốt với hệ thống truy xuất.

Llama là họ mô hình mã nguồn mở do Meta phát triển. Các phiên bản gần đây (Llama 3, Llama 3.1, Llama 3.2, Llama 3.3) đạt chất lượng gần các mô hình

thương mại. Ưu điểm của Llama và các mô hình mã nguồn mở là có thể tự triển khai trên hạ tầng riêng (on-premise), giảm chi phí và bảo vệ dữ liệu. Thông qua Ollama, vLLM, llama.cpp, các công cụ này có thể chạy trên một máy chủ có GPU phù hợp.

Các mô hình mã nguồn mở khác đáng chú ý: - Mistral (Mistral AI): các phiên bản 7B, 8x7B (MoE), Mistral Large. - DeepSeek (Trung Quốc): DeepSeek-V2, DeepSeek-V3, có chất lượng cao với chi phí thấp. - Qwen (Alibaba): Qwen 2, Qwen 2.5 với nhiều kích thước. - VinaLlama và PhoGPT (Việt Nam): các mô hình được tối ưu cho tiếng Việt.

c. Đặc điểm kỹ thuật phục vụ doanh nghiệp

Khi triển khai LLM trong môi trường doanh nghiệp, cần quan tâm đến các đặc điểm:

- Chi phí sử dụng: mỗi provider có chính sách giá khác nhau, tính theo token đầu vào/đầu ra. Cần cơ chế cache, lựa chọn model phù hợp cho từng tác vụ.
- Latency: thời gian phản hồi ảnh hưởng đến trải nghiệm người dùng, đặc biệt trong các ứng dụng thời gian thực.
- Quota và Rate limit: mỗi provider giới hạn số request mỗi phút. Hệ thống cần cơ chế retry, queue, fallback.
- Privacy và Data residency: doanh nghiệp có thể yêu cầu dữ liệu không rời khỏi hạ tầng (châu Âu GDPR, y tế, tài chính).
- Reproducibility: kết quả có thể thay đổi giữa các lần gọi do temperature > 0. Cần chiến lược đảm bảo tính ổn định.

1.1.3. Hạn chế của LLM trong bài toán doanh nghiệp

Mặc dù LLM mang lại nhiều bước tiến quan trọng, các mô hình này vẫn tồn tại nhiều hạn chế khi áp dụng vào hệ thống doanh nghiệp [6]:

(1) Hiện tượng ảo giác (hallucination): mô hình có thể tạo ra thông tin nghe có vẻ hợp lý nhưng không chính xác hoặc không có căn cứ. Trong bối cảnh doanh nghiệp, điều này đặc biệt nguy hiểm vì có thể dẫn đến quyết định sai lầm, thông tin khách hàng sai, hoặc vi phạm quy định.

(2) Phụ thuộc vào dữ liệu huấn luyện: chất lượng, phạm vi và thời điểm thu thập dữ liệu ảnh hưởng trực tiếp đến độ chính xác. Các mô hình có thể không biết về sản phẩm mới, chính sách nội bộ, hoặc thông tin cụ thể của doanh nghiệp.

(3) Không tự cập nhật tri thức: LLM không tự động nắm bắt các thông tin mới phát sinh sau thời điểm huấn luyện nếu không có cơ chế bổ sung dữ liệu bên ngoài. Trong doanh nghiệp, thông tin thay đổi liên tục (giá sản phẩm, tồn

kho, chính sách nhân sự...).

(4) Chi phí tính toán cao: việc huấn luyện và triển khai LLM đòi hỏi tài nguyên phần cứng lớn. Chi phí sử dụng API thương mại cũng tăng theo quy mô.

(5) Hạn chế khi xử lý ngữ cảnh dài: mô hình có thể gặp khó khăn khi đầu vào chứa nhiều tài liệu hoặc thông tin cần liên kết qua nhiều đoạn văn bản, mặc dù các phiên bản mới đã cải thiện đáng kể.

(6) Bảo mật và quyền riêng tư: khi sử dụng API thương mại, dữ liệu của doanh nghiệp có thể được gửi đến server bên thứ ba, gây lo ngại về bảo mật.

(7) Chi phí tích hợp cao: mỗi hệ thống, mỗi use case thường cần code tích hợp riêng, dẫn đến nhân bản công sức.

Do đó, trong các hệ thống doanh nghiệp, LLM thường được kết hợp với các kỹ thuật bổ trợ: Retrieval-Augmented Generation (RAG) để cung cấp tri thức cập nhật; AI Agent kết hợp tool calling để thực hiện tác vụ; MCP để chuẩn hóa giao tiếp với hệ thống ngoài. Các kỹ thuật này được trình bày trong các mục tiếp theo.

1.2. Retrieval-Augmented Generation (RAG)

1.2.1. Khái niệm và nguyên lý hoạt động

Retrieval-Augmented Generation (RAG) là phương pháp kết hợp mô hình ngôn ngữ lớn với cơ chế truy xuất thông tin từ nguồn dữ liệu bên ngoài [7], [8]. Thay vì phụ thuộc hoàn toàn vào tri thức được học trong quá trình huấn luyện, mô hình có thể tham chiếu thêm các tài liệu liên quan tại thời điểm xử lý truy vấn để hỗ trợ tạo câu trả lời.

Về bản chất, RAG tách biệt tương đối giữa khả năng sinh ngôn ngữ của LLM và nguồn tri thức được sử dụng. Tri thức có thể được lưu trữ, cập nhật và truy xuất từ bên ngoài mô hình, qua đó giúp hệ thống linh hoạt hơn trong các bài toán yêu cầu thông tin mới, thông tin chuyên ngành hoặc dữ liệu thay đổi theo thời gian.

Quy trình tổng quát của một hệ thống RAG gồm bốn bước chính:

Bước 1: Tiếp nhận truy vấn Người dùng gửi câu hỏi hoặc yêu cầu. Truy vấn có thể được chuẩn hóa, bổ sung ngữ cảnh và chuyển sang dạng phù hợp để phục vụ quá trình tìm kiếm (ví dụ: tạo embedding, tách thực thể).

Bước 2: Truy xuất tài liệu liên quan Retriever thực hiện tìm kiếm trong kho tri thức dựa trên nội dung truy vấn. Hệ thống lựa chọn các tài liệu hoặc đoạn văn bản có mức độ liên quan cao nhất. Kết quả truy xuất thường là một tập các đoạn thông tin ngắn phục vụ cho bước sinh câu trả lời. Có hai hướng truy xuất chính:

- Dense Retrieval: sử dụng mô hình embedding ánh xạ truy vấn và tài liệu vào

cùng không gian vector; mức độ liên quan đo bằng cosine similarity. - Sparse Retrieval: sử dụng thuật toán TF-IDF, BM25 [9]; đánh giá mức độ liên quan dựa trên tần suất từ khóa.

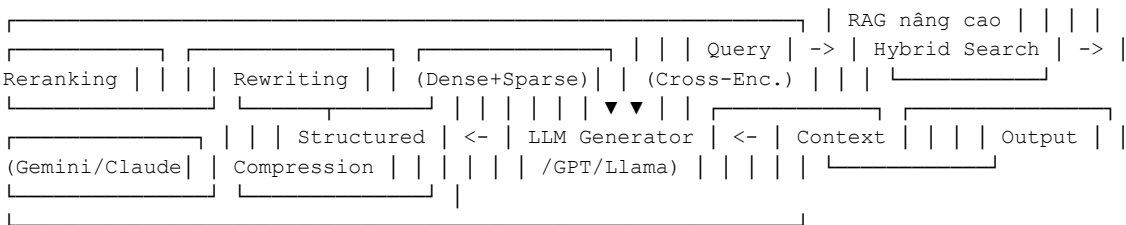
Bước 3: Xây dựng ngữ cảnh (Augmented Context) Các tài liệu được truy xuất được ghép với truy vấn ban đầu và các thông tin bổ sung (hồ sơ người dùng, dữ liệu có cấu trúc, thông tin ngữ cảnh khác) để tạo thành ngữ cảnh mở rộng. Giai đoạn này có thể áp dụng các kỹ thuật nén (context compression) nhằm giảm kích thước ngữ cảnh và giữ lại phần liên quan nhất.

Bước 4: Sinh câu trả lời (Generation) LLM sử dụng truy vấn cùng với ngữ cảnh mở rộng để tạo ra câu trả lời. Câu trả lời được xây dựng dựa trên cả kiến thức nội tại của mô hình và thông tin được truy xuất từ kho tri thức. Nhờ đó, đầu ra thường có độ chính xác và tính cập nhật cao hơn so với cách tiếp cận chỉ sử dụng LLM độc lập.

1.2.2. Kiến trúc nâng cao của RAG

Hệ thống RAG trong nghiên cứu và triển khai hiện đại thường được mở rộng với nhiều thành phần:

- Query Understanding: phân tích truy vấn, trích xuất thực thể, mở rộng truy vấn (query expansion, query rewriting) để cải thiện chất lượng truy xuất.
- Hybrid Search: kết hợp dense và sparse retrieval thông qua các thuật toán như Reciprocal Rank Fusion (RRF) để tận dụng ưu điểm bổ trợ.
- Reranking: sử dụng mô hình học sâu (Cross-Encoder) để đánh giá lại mức độ liên quan của từng tài liệu sau bước truy xuất ban đầu, sắp xếp lại thứ tự ưu tiên.
- Context Compression: nén ngữ cảnh truy xuất nhằm giảm kích thước đầu vào, loại bỏ thông tin nhiễu, tăng hiệu quả xử lý.
- Structured Output: buộc LLM sinh kết quả theo cấu trúc JSON/schema định sẵn để dễ dàng tích hợp với hệ thống.



1.2.3. Vai trò của RAG trong hệ thống doanh nghiệp

Trong các hệ thống ứng dụng thực tế, RAG đóng vai trò quan trọng trong việc mở rộng khả năng của LLM vượt ra ngoài giới hạn tri thức nội tại của mô hình [7], [8]. Đối với doanh nghiệp, RAG mang lại một số lợi ích chính:

- Bổ sung và cập nhật tri thức liên tục: thông qua kho dữ liệu bên ngoài mà không cần huấn luyện lại toàn bộ mô hình.
- Tăng khả năng kiểm soát và truy vết nguồn gốc thông tin: hệ thống có thể chỉ ra tài liệu nào đã được sử dụng, phục vụ tuân thủ và kiểm toán.
- Hỗ trợ cho các bài toán theo miền chuyên biệt: nhờ khả năng tích hợp dữ liệu riêng của từng doanh nghiệp.
- Tiết kiệm chi phí so với fine-tuning: trong nhiều trường hợp, việc tinh chỉnh một LLM lớn rất tốn kém, RAG cho phép đạt được kết quả tương đương với chi phí thấp hơn nhiều.

1.2.4. Ưu điểm và hạn chế của RAG

Bảng 1.1. Ưu điểm và hạn chế của RAG

Ưu điểm	Hạn chế
Mở rộng khả năng sử dụng tri thức ngoài mô hình, giảm phụ thuộc vào tham số đã huấn luyện	Phụ thuộc đáng kể vào chất lượng, độ chính xác và mức độ cập nhật của nguồn dữ liệu truy xuất
Cho phép cập nhật thông tin thông qua kho dữ liệu bên ngoài mà không cần huấn luyện lại mô hình	Hiệu quả đầu ra bị ảnh hưởng nếu bộ truy xuất chọn sai, thiếu hoặc nhiễu thông tin liên quan
Hỗ trợ tốt các bài toán theo miền nhờ khả năng tích hợp dữ liệu chuyên biệt	Làm tăng độ phức tạp trong thiết kế, triển khai và vận hành hệ thống so với LLM thuần túy
Giúp phản hồi bám sát hơn vào yêu cầu người dùng và tài liệu được truy xuất	Tăng thời gian xử lý do phát sinh thêm bước truy xuất trước khi sinh kết quả
Tăng khả năng kiểm soát và truy vết nguồn thông tin đầu ra	Cần thiết kế tốt cơ chế thu thập, tổ chức, chia đoạn và biểu diễn dữ liệu để đạt hiệu quả cao

Trong bối cảnh doanh nghiệp, RAG thường được kết hợp với các kỹ thuật tăng cường (Hybrid Search, Reranking, Compression) và được nhúng vào pipeline AI Agent để tạo ra các hệ thống thông minh có khả năng tự động hóa quy trình nghiệp vụ.

1.3. AI Agent và cơ chế phối hợp đa tác nhân (Multi-Agent)

1.3.1. Khái niệm AI Agent

AI Agent là một hệ thống phần mềm có khả năng tự chủ cảm nhận môi trường, lập kế hoạch, ra quyết định và thực hiện hành động để đạt được mục tiêu được giao, thông qua việc kết hợp LLM với các công cụ (tools) và bộ nhớ (memory) [10], [11].

Trong ngữ cảnh LLM, một AI Agent thường gồm các thành phần:

- LLM làm "bộ não": phân tích yêu cầu, lập kế hoạch và quyết định hành động tiếp theo.

- Tools (công cụ): các hàm API mà agent có thể gọi, ví dụ: tra cứu cơ sở dữ liệu, gọi API hệ thống ngoài, tạo báo cáo.
- Memory (bộ nhớ): lưu trữ ngữ cảnh hội thoại, kết quả trung gian, sở thích người dùng.
- Planning: khả năng phân tách tác vụ lớn thành các bước nhỏ và xử lý tuần tự hoặc song song.
- Reflection / Self-critique: khả năng tự đánh giá kết quả và điều chỉnh.

Các framework AI Agent phổ biến gồm: LangChain/LangGraph (Python, JS), AutoGen (Microsoft), CrewAI, LlamaIndex Agents, Semantic Kernel (.NET). Trong .NET, Semantic Kernel của Microsoft cung cấp abstractions cho AI Agent, function calling, planner và memory.

1.3.2. Kiến trúc tác nhân và cơ chế lập kế hoạch

Một kiến trúc agent phổ biến là ReAct (Reason + Act), trong đó agent luân phiên giữa hai bước: - Reason: suy nghĩ về tình huống hiện tại, lên kế hoạch hành động tiếp theo. - Act: thực hiện hành động (gọi tool, truy vấn dữ liệu).

Một số agent nâng cao hơn sử dụng: - Plan-and-Execute: lập kế hoạch toàn bộ trước, sau đó thực thi tuần tự. - Multi-step Reasoning: chain-of-thought, tree-of-thought để giải quyết vấn đề phức tạp. - Reflection: tự đánh giá và sửa chữa kết quả.

1.3.3. Multi-Agent Orchestration

Multi-Agent System (MAS) là hệ thống gồm nhiều agent phối hợp để giải quyết vấn đề phức tạp. Mỗi agent có vai trò chuyên biệt (researcher, planner, executor, critic, ...) và giao tiếp với nhau thông qua message passing hoặc chia sẻ context chung.

Các mô hình phối hợp phổ biến:

- Supervisor Pattern: một agent "supervisor" điều phối, các agent con xử lý tác vụ con.
- Hierarchical Pattern: agent cấp cao phân tách, agent cấp thấp thực thi.
- Collaborative Pattern: các agent trao đổi ngang hàng để đạt đồng thuận.
- Pipeline Pattern: agent A xử lý xong, chuyển output cho agent B tiếp tục.

Trong doanh nghiệp, Multi-Agent rất phù hợp cho các quy trình nghiệp vụ phức tạp: ví dụ một workflow "phê duyệt đơn nghỉ phép" có thể gồm: agent phân tích đơn → agent tra cứu HRM (số ngày phép còn lại) → agent gọi API để tạo ticket → agent gửi thông báo cho quản lý.

1.3.4. Ưu điểm và thách thức

Ưu điểm: - Giải quyết được các tác vụ phức tạp đa bước. - Phân tách trách nhiệm rõ ràng giữa các agent. - Dễ mở rộng (thêm agent mới khi có nhu cầu mới). - Tận dụng được tool calling để tương tác với hệ thống ngoài.

Thách thức: - Chi phí: nhiều agent có thể dẫn đến chi phí token cao. - Latency: thời gian phản hồi có thể tăng do nhiều bước LLM. - Error propagation: lỗi ở agent sớm có thể lan truyền. - Debugging: khó truy vết khi có nhiều agent phối hợp. - Đảm bảo tính an toàn: cần guardrail, human-in-the-loop cho các quyết định quan trọng.

Trong đề tài, Multi-Agent Orchestration được thiết kế kết hợp với MCP để tạo ra một hệ thống có khả năng phối hợp chặt chẽ với các hệ thống doanh nghiệp hiện hữu.

1.4. Giao thức Model Context Protocol (MCP)

1.4.1. Bối cảnh ra đời của MCP

Model Context Protocol (MCP) là một giao thức mở được Anthropic giới thiệu vào tháng 11 năm 2024 nhằm chuẩn hóa cách thức các mô hình AI (đặc biệt là LLM Agent) tương tác với các công cụ, dịch vụ và nguồn dữ liệu bên ngoài [12]. Trước khi MCP ra đời, mỗi framework AI Agent thường có cách tích hợp tools riêng, dẫn đến tình trạng phân mảnh: tool của framework A không sử dụng được cho framework B, doanh nghiệp muốn tích hợp tool mới phải code lại.

MCP giải quyết vấn đề này bằng cách định nghĩa một giao thức client-server chuẩn, tương tự như LSP (Language Server Protocol) trong lập trình:

- MCP Server: cung cấp các tool, resource và prompt qua giao diện chuẩn.
- MCP Client: có thể là AI Agent (Claude Desktop, Cursor IDE, các AI framework khác) kết nối tới MCP Server để sử dụng các khả năng được cung cấp.

Giao thức MCP sử dụng JSON-RPC 2.0 làm định dạng trao đổi và hỗ trợ hai cơ chế giao tiếp chính: stdio (cho local server) và HTTP+SSE (cho remote server).

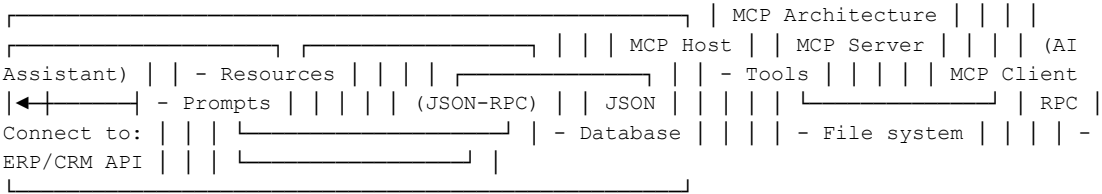
1.4.2. Kiến trúc và cách thức hoạt động

Kiến trúc MCP gồm ba lớp chính:

Lớp 1: MCP Host là ứng dụng AI (ví dụ: Claude Desktop, IDE, AI framework) muốn sử dụng các tool.

Lớp 2: MCP Client được host nhúng vào, có nhiệm vụ kết nối và giao tiếp với một hoặc nhiều MCP Server.

Lớp 3: MCP Server cung cấp ba loại khả năng: - Resources: dữ liệu mà LLM có thể đọc (file, database records, API responses). - Tools: hàm mà LLM có thể gọi để thực hiện hành động (tạo ticket, gửi email...). - Prompts: template prompt được định nghĩa sẵn cho các tác vụ cụ thể.



1.4.3. MCP trong bài toán tích hợp hệ thống doanh nghiệp

Đối với doanh nghiệp, MCP mang lại giá trị đặc biệt lớn:

(1) Chuẩn hóa giao tiếp AI – hệ thống: thay vì mỗi agent phải tích hợp riêng với từng hệ thống, doanh nghiệp chỉ cần triển khai một MCP Server cho mỗi hệ thống (ERP, CRM, DMS). Sau đó, bất kỳ AI Agent nào hỗ trợ MCP đều có thể kết nối.

(2) Tái sử dụng: tool khai thác từ MCP Server có thể được sử dụng bởi nhiều agent khác nhau, nhiều workflow khác nhau, thậm chí nhiều ứng dụng khác nhau.

(3) Bảo mật: MCP Server đặt tại hạ tầng doanh nghiệp, có thể áp dụng các chính sách kiểm soát truy cập, logging, audit. Dữ liệu nhạy cảm không cần gửi ra ngoài.

(4) Khả năng mở rộng: khi doanh nghiệp muốn tích hợp hệ thống mới, chỉ cần triển khai MCP Server tương ứng. Không cần sửa đổi AI Agent.

Đề tài sử dụng MCP như một cơ chế cốt lõi để nền tảng Enterprise AI có thể "Plug & Play" vào các hệ thống doanh nghiệp.

1.5. Tích hợp hệ thống doanh nghiệp (Enterprise System Integration)

1.5.1. Tổng quan về ERP, CRM, DMS, HRM

Enterprise Resource Planning (ERP): hệ thống hoạch định nguồn lực doanh nghiệp, tích hợp các quy trình cốt lõi như tài chính, kế toán, mua hàng, sản xuất, kho vận. Các sản phẩm phổ biến: SAP, Oracle ERP, Microsoft Dynamics, Odoo, ERPNext.

Customer Relationship Management (CRM): hệ thống quản lý quan hệ khách hàng, hỗ trợ quản lý leads, opportunities, sales pipeline, marketing automation. Ví dụ: Salesforce, HubSpot, Zoho CRM, Microsoft Dynamics CRM, custom CRM nội bộ.

Document Management System (DMS): hệ thống quản lý tài liệu, lưu trữ, phiên bản hóa, chia sẻ và tìm kiếm. Ví dụ: SharePoint, Alfresco, Nextcloud, hệ thống DMS nội bộ.

Human Resource Management (HRM): hệ thống quản lý nhân sự gồm hồ sơ nhân viên, chấm công, tính lương, đánh giá. Ví dụ: Workday, BambooHR, custom HRM.

Workflow / BPM: hệ thống quản lý quy trình nghiệp vụ (Business Process Management), điều phối luồng công việc giữa nhiều bên. Ví dụ: Camunda, jBPM, Activiti, custom BPM.

Business Intelligence (BI): hệ thống phân tích và báo cáo, ví dụ Power BI, Tableau, Metabase.

Database trực tiếp: nhiều hệ thống không có API chuẩn mà chỉ có database; tích hợp qua query SQL trực tiếp.

1.5.2. Các phương pháp tích hợp truyền thống

Trước khi AI trở nên phổ biến, doanh nghiệp đã có nhiều phương pháp tích hợp hệ thống:

- Point-to-Point: hai hệ thống kết nối trực tiếp, đơn giản cho cặp đôi nhưng khó mở rộng.
- Hub-and-Spoke: một hub trung gian, các hệ thống kết nối vào hub. Ví dụ: ESB (Enterprise Service Bus).
- Middleware/ESB: MuleSoft, IBM Integration Bus, WSO2.
- API Gateway: Kong, Apigee, AWS API Gateway.
- Message Queue: Kafka, RabbitMQ, ActiveMQ dùng cho tác vụ bất đồng bộ.

Các phương pháp này vẫn được sử dụng rộng rãi, đặc biệt khi tích hợp giữa các hệ thống nghiệp vụ truyền thống. Tuy nhiên, chúng chưa giải quyết được bài toán tích hợp AI: cần một lớp khác có khả năng hiểu ngôn ngữ tự nhiên, lập kế hoạch và gọi tool.

1.5.3. Thách thức khi tích hợp AI vào hệ thống doanh nghiệp

Khi tích hợp AI vào hệ thống doanh nghiệp, gặp phải các thách thức:

(1) Đa dạng hệ thống và API: mỗi hệ thống có API riêng, schema khác nhau, cần abstraction để AI Agent giao tiếp thống nhất.

(2) Quyền truy cập (Authorization): AI không thể có mọi quyền; cần cơ chế phân quyền chi tiết (RBAC/ABAC), giới hạn tool theo role.

(3) Audit và Compliance: mọi hành động của AI cần được ghi log để phục vụ kiểm toán và tuân thủ (GDPR, HIPAA, ISO 27001).

(4) Đảm bảo chất lượng đầu ra: AI có thể sinh ra câu trả lời sai; cần Human-in-the-Loop cho quyết định quan trọng.

(5) Chi phí & hiệu năng: việc gọi LLM nhiều lần tốn kém và chậm; cần cache, batching và chọn model phù hợp.

(6) Bảo mật & quyền riêng tư: dữ liệu nội bộ khi qua LLM có thể bị rò rỉ; cần giải pháp on-premise hoặc private cloud.

(7) Đồng bộ hóa tri thức: khi doanh nghiệp thay đổi quy trình, sản phẩm, kho tri thức phải được cập nhật.

Đề tài đề xuất một kiến trúc giải quyết đồng thời các thách thức trên, với MCP làm chuẩn giao tiếp, RAG làm cơ chế truy xuất tri thức và AI Gateway làm lớp kiểm soát bảo mật.

1.6. Các nghiên cứu và hệ thống liên quan

1.6.1. Các nền tảng AI doanh nghiệp hiện có

Hiện nay có một số nền tảng AI doanh nghiệp đáng chú ý:

Microsoft Copilot Studio: cho phép doanh nghiệp tạo các "Copilot" tùy chỉnh tích hợp với Microsoft 365, Dynamics 365, Power Platform. Ưu điểm: hệ sinh thái Microsoft rộng, bảo mật tốt. Hạn chế: gắn liền với Microsoft stack, chi phí cao.

Google Vertex AI Agent Builder: cho phép xây dựng agent tích hợp với Google Cloud, hỗ trợ nhiều LLM. Ưu điểm: linh hoạt, tích hợp GCP. Hạn chế: cần GCP.

Amazon Bedrock Agents: tương tự Vertex AI nhưng trên AWS. Hỗ trợ nhiều foundation model.

Salesforce Einstein: AI cho CRM Salesforce. Tích hợp chặt với Salesforce.

LangChain / LlamaIndex: framework mã nguồn mở, cung cấp primitives để xây dựng agent.

Bảng 1.2. So sánh một số nền tảng AI doanh nghiệp hiện có

Nền tảng	Đặc điểm	Hạn chế
Microsoft Copilot Studio	Tích hợp chặt với Microsoft 365, Dynamics; bảo mật cấp doanh nghiệp	Vendor lock-in Microsoft; chi phí license cao; khó tích hợp hệ thống ngoài hệ sinh thái

Google Vertex AI Agent Builder	Hỗ trợ đa model, tích hợp GCP, RAG built-in	Cần Google Cloud; giá cao; ít tooling cho môi trường on-premise
Amazon Bedrock Agents	Đa model foundation, tích hợp AWS, knowledge base	Cần AWS; phức tạp cho người mới
LangChain (mã nguồn mở)	Framework linh hoạt, cộng đồng lớn	Không phải sản phẩm hoàn chỉnh; tự triển khai mọi thứ
Salesforce Einstein	AI native cho CRM Salesforce	Gắn liền Salesforce; không tổng quát

1.6.2. Các nghiên cứu về AI Agent và Multi-Agent

Trong lĩnh vực nghiên cứu, các công trình gần đây đáng chú ý:

- AutoGen (Microsoft Research): framework Multi-Agent hỗ trợ hội thoại giữa các agent.
- CrewAI: framework tổ chức "crew" các agent theo vai trò.
- LangGraph: mở rộng LangChain cho workflow đồ thị.
- Semantic Kernel (Microsoft): SDK cho .NET để xây dựng AI Agent tích hợp với C# / .NET.

Bảng 1.3. Một số nghiên cứu ứng dụng AI Agent trong doanh nghiệp

Nghiên cứu / Framework	Hướng tiếp cận	Hạn chế
AutoGen [13]	Multi-Agent Conversation; phối hợp qua messages	Chưa có chuẩn MCP chính thức; khó debug khi agent nhiều
CrewAI [14]	Role-based Multi-Agent (researcher, writer...)	Giới hạn về workflow phức tạp
LangGraph	Workflow đồ thị có trạng thái, hỗ trợ ReAct, Reflection	Phụ thuộc LangChain; learning curve
Semantic Kernel (.NET)	Function calling, planner, memory cho .NET	Cộng đồng nhỏ hơn Python ecosystem

1.6.3. Khoảng trống nghiên cứu

Qua khảo sát, có thể nhận thấy các khoảng trống nghiên cứu sau:

(1) Thiếu kiến trúc tổng thể tích hợp nhiều công nghệ AI: các nghiên cứu thường tập trung vào một khía cạnh (RAG, agent, tool calling). Chưa có nhiều công trình đề xuất kiến trúc thống nhất, triển khai được thực tế cho doanh nghiệp.

(2) Thiếu hỗ trợ chuẩn MCP ở mức production-grade: MCP ra đời 2024, nhiều framework đang trong quá trình tích hợp. Một kiến trúc hoàn chỉnh kết hợp MCP với các thành phần khác còn hạn chế.

(3) Thiếu hướng dẫn triển khai từng bước (Phase): hầu hết tài liệu dừng ở mức kiến trúc, rất ít tài liệu hướng dẫn cụ thể cách triển khai từ prototype đến production với quy mô team 20-100 người.

(4) Thiếu đánh giá trên bối cảnh doanh nghiệp Việt Nam: các công trình chủ yếu ở Mỹ, châu Âu, Trung Quốc. Bối cảnh Việt Nam với đặc thù tiếng Việt, văn hóa doanh nghiệp, hạ tầng còn hạn chế chưa được nghiên cứu sâu.

(5) Thiếu giải pháp "Plug and Play" thực sự cho doanh nghiệp nhỏ và vừa: các giải pháp thương mại đòi hỏi đầu tư lớn, các framework mã nguồn mở đòi hỏi đội ngũ kỹ thuật cao.

Đề tài nhằm giải quyết các khoảng trống trên bằng cách đề xuất một kiến trúc Enterprise AI Platform hoàn chỉnh, có hướng dẫn triển khai từng phase, tích hợp MCP, RAG và AI Agent, phù hợp với doanh nghiệp Việt Nam.

1.7. Kết luận chương 1

Chương 1 đã trình bày nền tảng lý thuyết và tổng quan nghiên cứu của luận văn, bao gồm:

- Tổng quan về trí tuệ nhân tạo và mô hình ngôn ngữ lớn (LLM): khái niệm, các mô hình tiêu biểu (GPT, Claude, Gemini, Llama) và hạn chế khi áp dụng cho doanh nghiệp.
- Kỹ thuật Retrieval-Augmented Generation (RAG): khái niệm, kiến trúc tổng quát, kiến trúc nâng cao với Hybrid Search, Reranking, Context Compression và vai trò trong hệ thống doanh nghiệp.
- AI Agent và Multi-Agent Orchestration: khái niệm AI Agent, kiến trúc ReAct/Plan-and-Execute, các mô hình phối hợp Multi-Agent.
- Giao thức Model Context Protocol (MCP): bối cảnh ra đời, kiến trúc client-server và vai trò trong tích hợp hệ thống doanh nghiệp.
- Tích hợp hệ thống doanh nghiệp: tổng quan về ERP, CRM, DMS, HRM; các phương pháp tích hợp truyền thống; thách thức khi tích hợp AI.
- Khảo sát các nghiên cứu và hệ thống liên quan; xác định khoảng trống nghiên cứu cần giải quyết.

Trên cơ sở đó, chương tiếp theo sẽ tập trung vào phân tích bài toán, đối tượng nghiên cứu cụ thể và đề xuất kiến trúc hệ thống Enterprise AI Platform cho bài toán tích hợp đa hệ thống doanh nghiệp.

Chương 2: PHÂN TÍCH BÀI TOÁN VÀ THIẾT KẾ NỀN TẢNG

Chương này trình bày quá trình phân tích bài toán, xác định yêu cầu chức năng và phi chức năng, đề xuất kiến trúc tổng thể của nền tảng Enterprise AI Platform, thiết kế chi tiết các module cốt lõi, thiết kế dữ liệu và cơ sở tri thức, cùng quy trình triển khai theo 5 giai đoạn (Phase).

2.1. Phân tích bài toán

2.1.1. Mô tả bài toán và bối cảnh doanh nghiệp

Bài toán trung tâm của đề tài là: "Làm thế nào để xây dựng một nền tảng AI có khả năng tích hợp nhanh chóng vào nhiều hệ thống doanh nghiệp khác nhau, độc lập với AI provider, có khả năng mở rộng và đạt chuẩn doanh nghiệp về bảo mật, kiểm toán?"

Trong thực tế, doanh nghiệp hiện đại vận hành đồng thời nhiều hệ thống phục vụ các mục đích khác nhau: ERP để quản lý nguồn lực, CRM để quản lý khách hàng, DMS để quản lý tài liệu, HRM để quản lý nhân sự, Workflow để điều phối quy trình. Mỗi hệ thống có dữ liệu riêng, API riêng và cơ chế bảo mật riêng. Khi muốn tích hợp AI, doanh nghiệp thường phải xây dựng từ đầu cho từng hệ thống, dẫn đến chi phí cao, thời gian dài và khó bảo trì.

Ví dụ cụ thể: một doanh nghiệp sản xuất sử dụng hệ thống ERP (quản lý đơn hàng, tồn kho), CRM (quản lý khách hàng, leads), DMS (lưu trữ hợp đồng, tài liệu pháp lý). Khi muốn có một trợ lý AI giúp nhân viên tra cứu thông tin, phê duyệt đơn hàng, tạo báo cáo tổng hợp từ nhiều nguồn, doanh nghiệp thường phải: - Tích hợp AI vào ERP (1 lần) - Tích hợp AI vào CRM (1 lần) - Tích hợp AI vào DMS (1 lần) - Tích hợp AI giữa các hệ thống (1 lần nữa)

Mỗi lần tích hợp đều yêu cầu code riêng, cấu hình riêng, testing riêng. Chi phí ước tính cho mỗi lần tích hợp thường từ 50-200 triệu đồng và mất 2-6 tháng. Với 5 hệ thống, tổng chi phí có thể lên đến hàng tỷ đồng.

Nền tảng Enterprise AI Platform đề xuất một kiến trúc trung gian cho phép: - Một lần cài đặt: triển khai nền tảng AI một lần duy nhất - Nhiều lần kết nối: mỗi hệ thống (ERP, CRM, DMS, HRM) được kết nối qua giao thức chuẩn (MCP) - Chia sẻ tri thức: kho tri thức doanh nghiệp chung, các agent có thể truy xuất từ bất kỳ hệ thống nào - Độc lập provider: có thể chọn bất kỳ LLM nào (OpenAI, Anthropic, Google, Ollama...) mà không cần thay đổi code tích hợp - Multi-tenant: nhiều doanh nghiệp cùng sử dụng một nền tảng với dữ liệu và cấu hình riêng biệt

2.1.2. Yêu cầu chức năng

Từ bài toán đã xác định, hệ thống cần đáp ứng các yêu cầu chức năng chính được trình bày trong Bảng 2.1.

Bảng 2.1. Yêu cầu chức năng của hệ thống

STT	Yêu cầu chức năng	Mô tả
1	Tương tác hội thoại bằng ngôn ngữ tự nhiên	Cho phép người dùng trao đổi với hệ thống bằng tiếng Việt/tiếng Anh để yêu cầu tác vụ, hỏi thông tin hoặc điều chỉnh yêu cầu qua nhiều lượt hội thoại
2	Truy xuất tri thức doanh nghiệp (RAG)	Tìm kiếm và truy xuất thông tin liên quan từ kho tri thức doanh nghiệp (tài liệu, database, API) để hỗ trợ trả lời chính xác, có căn cứ
3	Tích hợp đa hệ thống qua MCP	Kết nối được với nhiều hệ thống doanh nghiệp (ERP, CRM, DMS, HRM, Workflow) thông qua giao thức chuẩn MCP
4	AI Agent đa tác vụ	Agent có khả năng lập kế hoạch, phân tách tác vụ phức tạp thành các bước nhỏ, gọi tool để thực thi trên các hệ thống
5	Multi-Agent Orchestration	Điều phối nhiều agent chuyên biệt phối hợp giải quyết vấn đề, ví dụ: agent nghiên cứu + agent lập kế hoạch + agent thực thi
6	Gọi tool và function calling	Hệ thống có thể định nghĩa và gọi các tool (API call, SQL query, file read...) dưới sự kiểm soát của Agent Executor
7	Quản lý ngữ cảnh và bộ nhớ	Lưu trữ ngữ cảnh hội thoại (short-term memory), hồ sơ người dùng (long-term memory) phục vụ cá nhân hóa
8	Đa provider AI	Có khả năng kết nối và sử dụng đồng thời nhiều LLM provider (OpenAI, Anthropic, Google, Ollama...) thông qua abstraction layer
9	Quản lý đa tenant	Nhiều doanh nghiệp cùng sử dụng một nền tảng với dữ liệu, cấu hình, quyền truy cập riêng biệt
10	Plugin SDK và mở rộng	Cho phép nhà phát triển tạo plugin mới để mở rộng khả năng của nền tảng mà không cần sửa mã nguồn cốt lõi
11	Giao diện API chuẩn	Cung cấp REST API và/hoặc gRPC để tích hợp với hệ thống bên ngoài, hỗ trợ streaming cho phản hồi real-time

12	Quản lý quyền truy cập và audit	Xác thực người dùng (AuthN), phân quyền chi tiết (RBAC/ABAC), ghi log kiểm toán cho mọi tác vụ
----	---------------------------------	--

2.1.3. Yêu cầu phi chức năng

Bên cạnh các chức năng chính, hệ thống cần đáp ứng các yêu cầu phi chức năng nhằm đảm bảo khả năng vận hành ổn định, an toàn và phù hợp với bối cảnh doanh nghiệp.

Bảng 2.2. Yêu cầu phi chức năng của hệ thống

STT	Yêu cầu	Mô tả
1	Hiệu năng	Hệ thống phản hồi trong thời gian hợp lý: < 3 giây cho query RAG thông thường, < 10 giây cho workflow multi-agent. Hỗ trợ ít nhất 100 concurrent users trong một tenant
2	Khả năng mở rộng	Kiến trúc hỗ trợ mở rộng ngang (scale out) khi số người dùng, số request tăng lên. Các module độc lập có thể scale riêng
3	Độ tin cậy	Tỷ lệ uptime $\geq 99.5\%$ trong giờ hành chính. Khi một provider AI gặp sự cố, hệ thống tự động chuyển sang provider dự phòng (failover)
4	Bảo mật	Mọi dữ liệu được mã hóa khi truyền (TLS 1.3) và khi lưu trữ (AES-256). Phân tách dữ liệu giữa các tenant (Row-Level Security). Tuân thủ các tiêu chuẩn bảo mật phù hợp
5	Giám sát và quan sát	Mỗi thành phần phát ra log, metric, trace. Tích hợp OpenTelemetry. Có dashboard giám sát tập trung
6	Khả năng bảo trì	Kiến trúc module hóa, mỗi module có thể được nâng cấp, sửa lỗi độc lập mà không ảnh hưởng hệ thống tổng thể. Plugin có thể load/unload nóng (hot-reload)
7	Tính nhất quán dữ liệu	Dữ liệu giữa PostgreSQL (quan hệ), pgvector (vector), Redis (cache) cần được đồng bộ hợp lý

8	Khả năng tương thích ngược	Khi nâng cấp API, đảm bảo tương thích ngược với các phiên bản cũ của client trong ít nhất 6 tháng
---	----------------------------	---

2.2. Kiến trúc tổng thể

2.2.1. Nguyên tắc thiết kế và triết lý kiến trúc

Nền tảng Enterprise AI Platform được thiết kế theo 8 nguyên tắc cốt lõi, phản ánh triết lý kiến trúc hướng tới mục tiêu "cắm vào là chạy" và độc lập với công nghệ.

TẮC THIẾT KẾ	8 NGUYÊN
PLAY Cắm vào là chạy - Zero config, auto-detect	1. PLUG &
thuộc AI provider cụ thể	2. PROVIDER AGNOSTIC Không phụ
3. DOMAIN AWARE Hiểu business domain của hệ thống tích hợp	3. DOMAIN AWARE Hiểu business domain của hệ thống tích hợp
4. EVENT DRIVEN Kiến trúc hướng sự kiện, loosely coupled	4. EVENT DRIVEN Kiến trúc hướng sự kiện, loosely coupled
5. AGENT NATIVE Hỗ trợ AI Agent đa nhiệm từ đầu	5. AGENT NATIVE Hỗ trợ AI Agent đa nhiệm từ đầu
6. OFFLINE FIRST Ưu tiên local AI, hybrid cloud	6. OFFLINE FIRST Ưu tiên local AI, hybrid cloud
7. ENTERPRISE GRADE Bảo mật, audit, compliance cấp doanh nghiệp	7. ENTERPRISE GRADE Bảo mật, audit, compliance cấp doanh nghiệp
8. MULTI-TENANT Hỗ trợ nhiều tenant với config riêng	8. MULTI-TENANT Hỗ trợ nhiều tenant với config riêng

PLUG & PLAY: Nền tảng được thiết kế để khi một doanh nghiệp muốn kết nối hệ thống ERP mới, họ chỉ cần triển khai một MCP Server tương ứng và đăng ký với nền tảng. Không cần thay đổi code ở tầng AI, không cần cấu hình phức tạp. Auto-detect cho phép hệ thống tự nhận biết hệ thống mới khi được kết nối.

PROVIDER AGNOSTIC: Mọi tương tác với LLM đi qua lớp Provider Abstraction. Các service ở tầng trên (RAG, Agent, Gateway) không bao giờ gọi trực tiếp API của OpenAI hay Anthropic. Việc chuyển đổi provider chỉ cần thay đổi cấu hình, không cần thay đổi code nghiệp vụ.

DOMAIN AWARE: Nền tảng được thiết kế để hiểu ngữ cảnh nghiệp vụ của hệ thống được tích hợp. Mỗi connector có domain knowledge riêng (ví dụ: ERP connector hiểu "đơn hàng", "tồn kho"; CRM connector hiểu "lead", "opportunity").

EVENT DRIVEN: Các module giao tiếp chủ yếu qua event bus (in-process hoặc message queue), giảm sự phụ thuộc trực tiếp và tăng khả năng mở rộng.

AGENT NATIVE: Từ đầu, kiến trúc đã được thiết kế cho Multi-Agent. Mỗi agent có vai trò, tool và memory riêng. Agent Orchestration Layer cung cấp primitives cho việc điều phối.

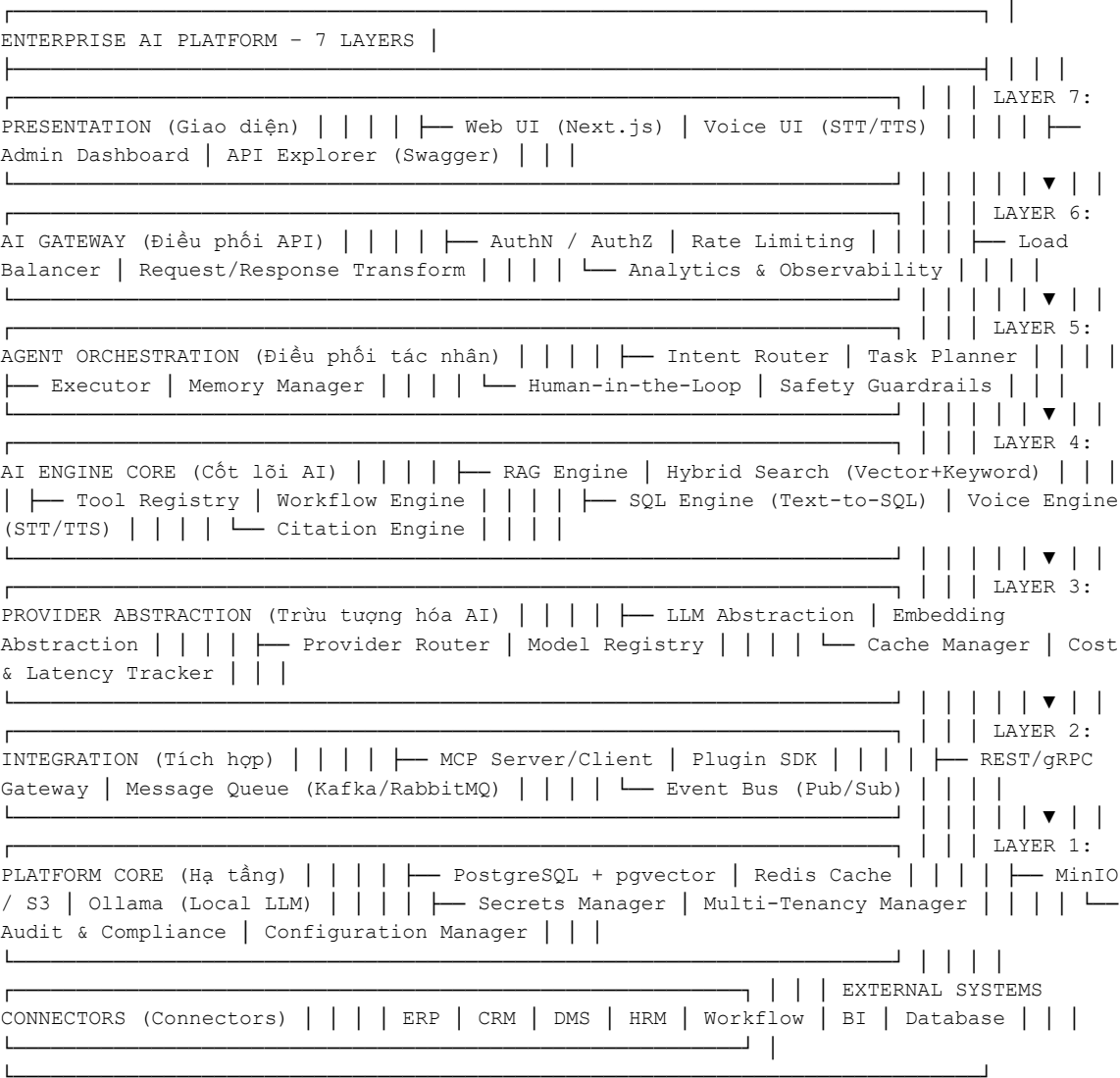
OFFLINE FIRST: Hệ thống ưu tiên chạy các mô hình local (Ollama) để đảm bảo dữ liệu không rời khỏi hạ tầng doanh nghiệp. Khi cần, có thể bổ sung cloud provider cho các tác vụ phức tạp hơn.

ENTERPRISE GRADE: Mọi thành phần đều có audit log, trace ID, rate limiting, circuit breaker. Secrets được quản lý qua Secrets Manager. RBAC + ABAC cho phép phân quyền chi tiết.

MULTI-TENANT: Kiến trúc cho phép nhiều doanh nghiệp cùng sử dụng một instance của nền tảng với cơ chế phân tách dữ liệu chặt chẽ (RLS - Row-Level Security trong PostgreSQL).

2.2.2. Kiến trúc 7 tầng (Layered Architecture)

Hệ thống được thiết kế theo kiến trúc phân lớp với 7 tầng rõ ràng, mỗi tầng có trách nhiệm và ranh giới riêng biệt.



Bảng 2.3. Mô tả chi tiết 7 Layer

Layer	Tên	Trách nhiệm	Ví dụ module
7	Presentation	Giao diện người dùng, API explorer	Web (Next.js), Voice (STT/TTS), Admin Dashboard

6	AI Gateway	Xác thực, phân quyền, rate limiting, routing, monitoring	AuthN/AuthZ, Rate Limiter, Load Balancer
5	Agent Orchestration	Điều phối agent, lập kế hoạch, thực thi	Intent Router, Task Planner, Executor, HITL
4	AI Engine Core	Xử lý AI cốt lõi	RAG, Hybrid Search, Tool Registry, Workflow, Memory
3	Provider Abstraction	Chuẩn hóa giao tiếp LLM/Embedding	LLM Provider, Embedding Provider, Router, Cache
2	Integration	Kết nối hệ thống ngoài	MCP Server, Plugin SDK, REST/gRPC, Event Bus
1	Platform Core	Hạ tầng, lưu trữ, bảo mật	PostgreSQL/pgvector, Redis, Secrets, Multi-tenancy

2.2.3. So sánh các phong cách kiến trúc

Bảng 2.4. So sánh Monolith vs Modular Monolith vs Microservice

Tiêu chí	Monolith	Modular Monolith	Microservice
Triển khai	Single unit	Single unit + modules	Multiple units
Coupling	Tight coupled	Loosely coupled	Independent
Linh hoạt công nghệ	Thấp	Cao	Cao nhất
Quyền tự chủ team	Thấp	Cao	Cao nhất
Độ phức tạp	Thấp	Trung bình	Cao
Testing	Dễ	Trung bình	Phức tạp
CI/CD	Đơn giản	Vừa phải	Phức tạp
Scale	Vertical only	Cả hai	Cả hai + granular
Phù hợp cho	< 20 devs	20-100 devs	> 100 devs
Thời gian khởi động	Nhanh	Trung bình	Chậm
Chi phí hạ tầng	Hiệu quả	Hiệu quả	Overhead

Quyết định kiến trúc: Đề tài lựa chọn chiến lược Modular Monolith (đầu) → Microservice (khi scale). Lý do:

- Giai đoạn đầu (Phase 1-3): team nhỏ (3-10 người), Modular Monolith cho phép phát triển nhanh, dễ quản lý, dễ debug. Mỗi module có interface rõ ràng, có thể test độc lập.
- Khi quy mô tăng lên (Phase 4-5 trở đi): khi số người dùng và request tăng, hoặc khi cần scale độc lập từng module, có thể tách module thành service riêng

mà không thay đổi interface.

- Chiến lược này được gọi là "Microservices when you need them" — bắt đầu đơn giản, phức tạp hóa khi cần.

2.3. Thiết kế các module cốt lõi

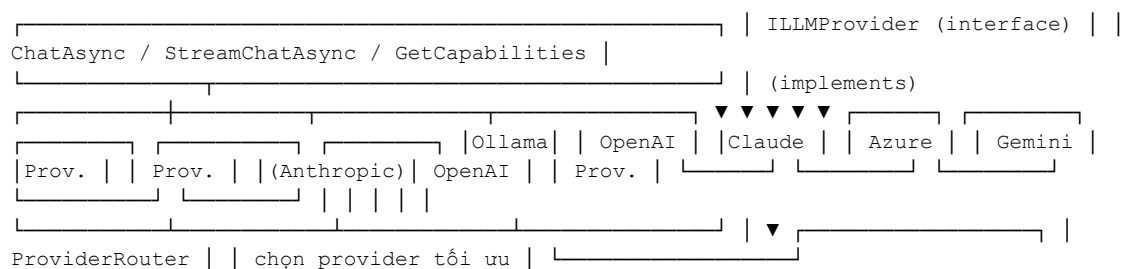
2.3.1. AI Provider Abstraction Layer

Layer này là "bí mật cốt lõi" của nền tảng — cho phép chuyển đổi giữa các AI provider mà không ảnh hưởng đến tầng trên. Toàn bộ các module khác chỉ phụ thuộc vào abstraction, không bao giờ gọi trực tiếp OpenAI/Anthropic API.

ILLMProvider Interface (C#):

```
public interface ILLMProvider { string ProviderId { get; } // "openai", "ollama", ...
Task ChatAsync(ChatRequest request, CancellationTokens ct); IEnumerable
StreamChatAsync(ChatRequest request, CancellationTokens ct); ModelCapabilities
GetCapabilities(string modelId); Task HealthCheckAsync(CancellationTokens ct); } public
record ChatRequest( string ModelId, IList Messages, double Temperature = 0.7, int?
MaxTokens = null, IReadOnlyDictionary? Tools = null, ToolChoice? ToolChoice = null, Guid
TenantId = default, string? TraceId = null ); public record ChatResponse( string Content,
string? FinishReason, UsageInfo Usage, IReadOnlyList? ToolCalls, string ProviderId,
string ModelId, TimeSpan Latency );
```

Các implementation cụ thể:



Provider Router: chọn provider tối ưu dựa trên: - Latency hiện tại của từng provider - Cost per token - Model capabilities phù hợp với tác vụ - User quota và rate limit - Cấu hình tenant (ưu tiên local Ollama hay cloud)

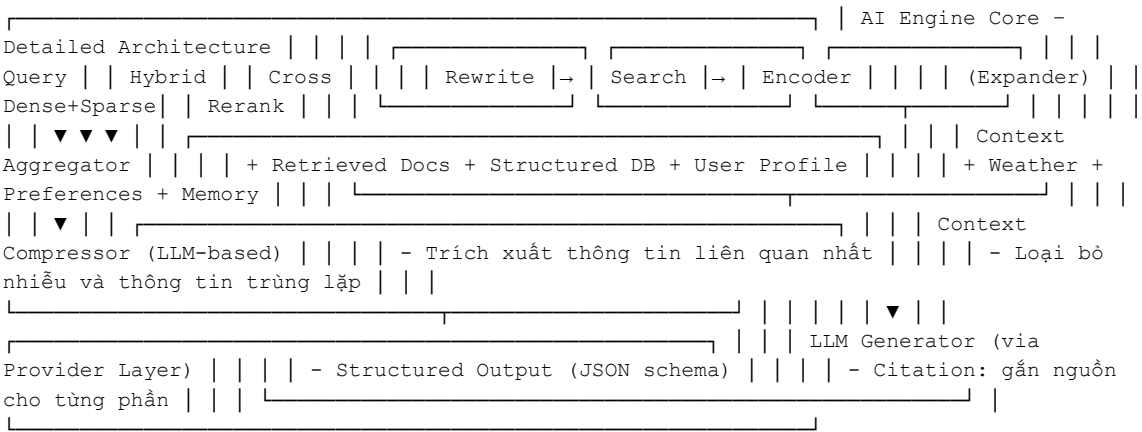
Embedding Provider: tương tự ILLMProvider nhưng cho embedding model. Cung cấp interface IEmbeddingProvider:

```
public interface IEmbeddingProvider { string ProviderId { get; } Task
EmbedAsync(EmbeddingRequest request, CancellationTokens ct); Task EmbedBatchAsync(IList
texts, CancellationTokens ct); } public record EmbeddingRequest(string Text, int?
Dimensions = null, Guid TenantId = default); public record
EmbeddingResponse(IReadOnlyList Vector, string ModelId, int Tokens, TimeSpan Latency);
```

Model Registry: catalog tất cả model được hỗ trợ với metadata: - Model ID, tên, nhà cung cấp - Capabilities (supports streaming, tool calling, vision...) - Context window size - Pricing (input/output token) - Rate limits

2.3.2. AI Engine Core – RAG Engine và Hybrid Search

AI Engine Core là thành phần cốt lõi xử lý truy xuất tri thức và sinh phản hồi. Kiến trúc RAG được thiết kế theo hướng Advanced RAG với nhiều tầng tối ưu.



Hybrid Search: kết hợp hai phương pháp truy xuất: - Dense Retrieval (Vector search): sử dụng embedding model chuyển truy vấn và tài liệu thành vector. Tìm kiếm dựa trên cosine similarity. - Sparse Retrieval (Keyword search): sử dụng BM25, đánh giá mức độ liên quan dựa trên tần suất từ khóa. Phù hợp với tên riêng, thuật ngữ kỹ thuật.

Kết quả từ hai phương pháp được hợp nhất bằng thuật toán Reciprocal Rank Fusion (RRF):

$$RRF(d) = \sum (1 / (k + rank_r(d)))$$

Cross-Encoder Reranking: sau khi hợp nhất, top N kết quả được đánh giá lại bằng Cross-Encoder — mô hình học sâu đánh giá trực tiếp cặp (query, document) để cho điểm relevance chính xác hơn.

Vector Store: sử dụng pgvector (PostgreSQL extension) hoặc có thể thay thế bằng Qdrant/Milvus/Chroma khi cần scale. pgvector được chọn cho Phase 1-2 vì: - Dùng chung PostgreSQL với dữ liệu quan hệ → đơn giản hóa vận hành - Hỗ trợ HNSW index cho tìm kiếm láng giềng gần đúng nhanh - ACID transaction đảm bảo nhất quán dữ liệu - Dễ dàng migrate sang specialized vector DB khi cần

Document Ingestion Pipeline: 1. Nhận tài liệu (PDF, DOCX, HTML, Markdown, CSV) 2. Parse và trích xuất nội dung (Apache Tika, unstructured library) 3. Chia nhỏ thành chunk (recursive character split, semantic chunking) 4. Tạo metadata (source, page, tenant_id, category) 5. Tạo embedding cho mỗi chunk 6. Lưu vào vector store kèm metadata

2.3.3. Agent Orchestration Layer

Agent Orchestration Layer là thành phần điều phối các AI Agent, chịu trách nhiệm phân tích ý định, lập kế hoạch, thực thi và quản lý ngữ cảnh.

Các thành phần chính:

Intent Router: phân loại ý định của người dùng từ input text. Kết quả đầu ra gồm: - Intent: loại tác vụ (hỏi đáp, tạo báo cáo, thực thi nghiệp vụ, hội thoại thông thường) - Entities: các thực thể được trích xuất (tên khách hàng, mã đơn hàng, ngày tháng...) - Confidence: độ tin cậy của phân loại

```
public interface IIntentRouter { Task ClassifyAsync(string userInput, Guid tenantId, CancellationTokens ct); Task> SuggestIntentsAsync(string userInput, Guid tenantId, CancellationTokens ct); }
```

Task Planner: phân tách một tác vụ phức tạp thành các bước (sub-tasks) có thể thực thi. Hỗ trợ: - Sequential: thực thi từng bước theo thứ tự - Parallel: thực thi song song các bước không phụ thuộc nhau - Conditional: thực thi bước tiếp theo dựa trên kết quả bước trước

```
public record TaskPlan( string PlanId, IReadOnlyList Steps, TaskPlanStatus Status, DateTime CreatedAt ); public record SubTask( string TaskId, string Description, SubTaskStatus Status, IReadOnlyList RequiredTools, string? AgentRole = null );
```

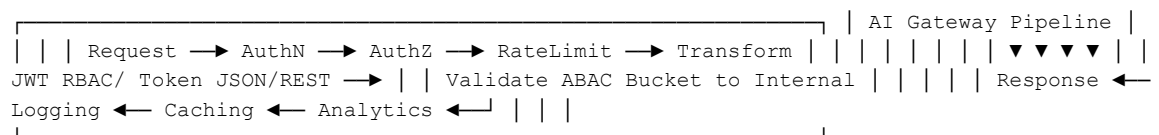
Agent Executor: thực thi plan theo từng bước. Mỗi bước: 1. Chọn tool phù hợp từ Tool Registry 2. Gọi tool, nhận kết quả 3. Ghi log kết quả vào Memory 4. Quyết định bước tiếp theo hoặc kết thúc

Safety Guardrails: kiểm tra đầu vào và đầu ra: - Input: kiểm tra prompt injection, PII (thông tin cá nhân nhạy cảm) - Output: kiểm tra nội dung không phù hợp, hallucination detection - Tool execution: kiểm tra quyền trước khi gọi tool nhạy cảm

Human-in-the-Loop (HITL): cho phép con người phê duyệt hoặc từ chối các quyết định quan trọng của agent trước khi thực thi. Ví dụ: khi agent muốn gửi email cho khách hàng, cần người quản lý xác nhận trước.

2.3.4. AI Gateway Layer

AI Gateway đóng vai trò "lễ tân" của toàn bộ nền tảng — tiếp nhận mọi request, kiểm tra bảo mật, điều phối và giám sát.



Authentication (AuthN): xác thực người dùng qua JWT token. Hỗ trợ: - Username/password (BCrypt hash) - SSO (tích hợp với Identity Provider qua OIDC) - API Key cho system-to-system

Authorization (AuthZ): phân quyền chi tiết dựa trên RBAC + ABAC: - RBAC: vai trò (admin, user, operator, viewer) → permissions - ABAC: policies dựa trên attributes (tenant_id, department, data_sensitivity)

Rate Limiter: giới hạn số request theo: - Per user (token limit) - Per tenant (request/minute) - Per IP (brute force protection) - Sử dụng Token Bucket algorithm với Redis

Analytics & Observability: tích hợp OpenTelemetry: - Traces: request ID propagation qua tất cả layers - Metrics: latency, token usage, error rate, queue depth - Logs: structured JSON logs với Serilog

2.3.5. Integration Layer – MCP và Plugin SDK

Integration Layer là cầu nối giữa nền tảng AI và các hệ thống doanh nghiệp bên ngoài.

MCP Server Implementation: nền tảng đóng vai trò MCP Server, cung cấp:

```
// MCP Server resource types public record ToolDefinition( string Name, string Description, JsonSchema InputSchema, bool IsDangerous, IReadOnlyList<string> RequiredPermissions ); public record ToolExecution( string ToolName, JsonObject Arguments, Guid ExecutionId, Guid TenantId, Guid UserId, DateTime ExecutedAt );
```

Mỗi connector (ERP, CRM, DMS...) được đăng ký với MCP Server như một tập tools. Ví dụ: - ERP Connector cung cấp: get_order(id), list_inventory(), create_purchase_request() - CRM Connector cung cấp: search_leads(query), update_contact(id, data), create_opportunity(data) - DMS Connector cung cấp: search_documents(query), get_document(id), list_versions(doc_id)

Plugin SDK: cho phép nhà phát triển mở rộng nền tảng mà không cần sửa mã nguồn cốt lõi.

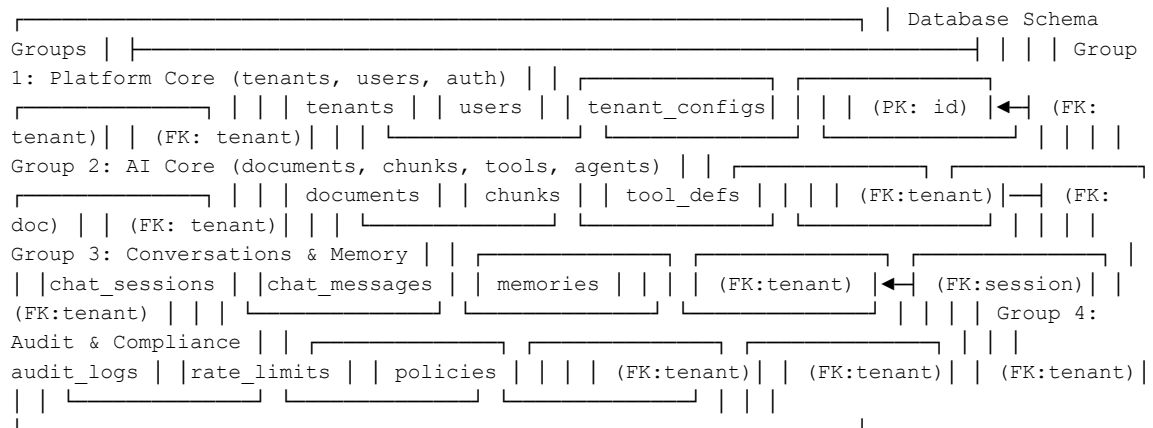
```
// IPlugin interface public interface IPlugin { string Id { get; } // Unique plugin ID string Version { get; } // Semantic version string Name { get; } // Human-readable name void Initialize(); // Called on load void Configure(PluginConfig config); void Dispose(); // Called on unload } // Plugin types public interface IAgentPlugin : IPlugin { IReadOnlyList<string> GetAgentRoles(); Task ExecuteAsync(AgentContext context); } public interface IToolPlugin : IPlugin { IReadOnlyList<string> GetTools(); Task ExecuteToolAsync(string toolName, JsonObject args, ToolContext ctx); }
```

Plugin được load qua PluginLoader với: - Hot-reload: plugin mới có thể được thêm mà không cần restart - Version checking: đảm bảo tương thích version - Sandboxing: plugin chạy trong AppDomain/process riêng với security restrictions - Discovery: tự động phát hiện plugin mới qua convention (thư mục /plugins hoặc config)

2.4. Thiết kế dữ liệu và cơ sở tri thức

2.4.1. Mô hình dữ liệu quan hệ (PostgreSQL)

Hệ thống sử dụng PostgreSQL 16 làm cơ sở dữ liệu quan hệ chính, tích hợp pgvector cho dữ liệu vector. Lược đồ dữ liệu được thiết kế theo bốn nhóm chức năng chính.



Nhóm Platform Core: - tenants: thông tin doanh nghiệp (id, name, config, plan_tier, created_at) - users: tài khoản người dùng (id, tenant_id, email, password_hash, role, status) - user_sessions: phiên đăng nhập (id, user_id, jwt_id, created_at, expires_at)

Nhóm AI Core: - documents: tài liệu đã upload (id, tenant_id, title, source_type, metadata, created_at) - chunks: đoạn nhỏ của tài liệu sau khi chia (id, document_id, content, metadata, created_at) - tool_definitions: định nghĩa tool cho agent (id, tenant_id, name, description, input_schema, permissions) - tool_calls: lịch sử gọi tool (id, tool_def_id, arguments, result, execution_time, user_id)

Nhóm Hội thoại & Memory: - chat_sessions: phiên hội thoại (id, tenant_id, user_id, title, created_at) - chat_messages: tin nhắn trong phiên (id, session_id, role, content, model_id, usage_info) - memories: bộ nhớ dài hạn của agent (id, tenant_id, user_id, memory_type, content, importance_score)

Nhóm Audit: - audit_logs: log kiểm toán (id, tenant_id, user_id, action, resource, details, ip_address, timestamp) - rate_limit_buckets: bucket giới hạn tốc độ (key, type, used, reset_at)

Bảng 2.5. Tổng hợp các bảng dữ liệu chính

Bảng	Số trường	Mô tả	Quan hệ chính
tenants	8	Thông tin doanh nghiệp	1:N users, 1:N configs
users	12	Tài khoản người dùng	N:1 tenants, 1:N sessions
documents	10	Tài liệu đã upload	N:1 tenants, 1:N chunks
chunks	8	Đoạn tài liệu	N:1 documents
chat_sessions	7	Phiên hội thoại	N:1 users, 1:N messages
chat_messages	10	Tin nhắn	N:1 sessions
memories	9	Bộ nhớ agent	N:1 users, N:1 tenants

tool_definitions	10	Định nghĩa tool	N:1 tenants
audit_logs	12	Log kiểm toán	N:1 tenants, N:1 users

2.4.2. Cơ sở dữ liệu vector (pgvector)

pgvector được sử dụng để lưu trữ và truy xuất vector embedding. Dữ liệu vector được lưu trong bảng chunks với trường embedding:

```
CREATE EXTENSION IF NOT EXISTS vector; CREATE TABLE chunks ( id UUID PRIMARY KEY DEFAULT gen_random_uuid(), document_id UUID NOT NULL REFERENCES documents(id) ON DELETE CASCADE, content TEXT NOT NULL, embedding vector(1536), -- Gemini Embedding: 1536 dims, OpenAI: 1536, text-embedding-3-small: 1536 metadata JSONB DEFAULT '{}', tenant_id UUID NOT NULL REFERENCES tenants(id), created_at TIMESTAMP DEFAULT NOW(), updated_at TIMESTAMP DEFAULT NOW() ); -- Index HNSW cho tìm kiếm vector nhanh CREATE INDEX idx_chunks_embedding_hnsw ON chunks USING hnsw (embedding vector_cosine_ops) WITH (m = 16, ef_construction = 64); -- Index cho filter theo tenant CREATE INDEX idx_chunks_tenant ON chunks (tenant_id);
```

Việc dùng chung PostgreSQL cho cả dữ liệu quan hệ và vector giúp: - Đơn giản hóa vận hành (chỉ 1 database) - Dễ backup/restore - JOIN giữa dữ liệu quan hệ và vector trong cùng query - Transaction ACID cho cả hai loại dữ liệu

2.4.3. Multi-tenant với Row-Level Security (RLS)

Cơ chế multi-tenant được triển khai với Row-Level Security (RLS) của PostgreSQL, đảm bảo dữ liệu giữa các tenant hoàn toàn tách biệt.

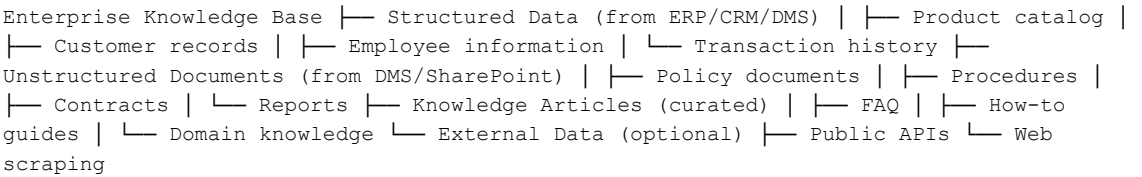
```
-- Enable RLS on all tenant-scoped tables ALTER TABLE users ENABLE ROW LEVEL SECURITY; ALTER TABLE documents ENABLE ROW LEVEL SECURITY; ALTER TABLE chunks ENABLE ROW LEVEL SECURITY; ALTER TABLE chat_sessions ENABLE ROW LEVEL SECURITY; -- Policy: user chỉ thấy dữ liệu của tenant của mình CREATE POLICY tenant_isolation ON users USING (tenant_id = current_setting('app.current_tenant_id')::uuid); CREATE POLICY tenant_isolation ON documents USING (tenant_id = current_setting('app.current_tenant_id')::uuid);
```

Application layer thiết lập tenant context cho mỗi request qua SET app.current_tenant_id = '...'. Mọi query tự động bị filter theo tenant, ngay cả khi developer quên thêm điều kiện WHERE.

Bổ sung: tenant config cho phép tùy biến: - Mô hình LLM mặc định - Cấu hình RAG (số chunks trả về, compression ratio) - Rate limit per user - Bật/tắt các plugin - Branding giao diện

2.4.4. Enterprise Knowledge Base

Kho tri thức doanh nghiệp (Enterprise Knowledge Base) là nguồn dữ liệu đầu vào cho RAG. Dữ liệu được tổ chức theo cấu trúc:



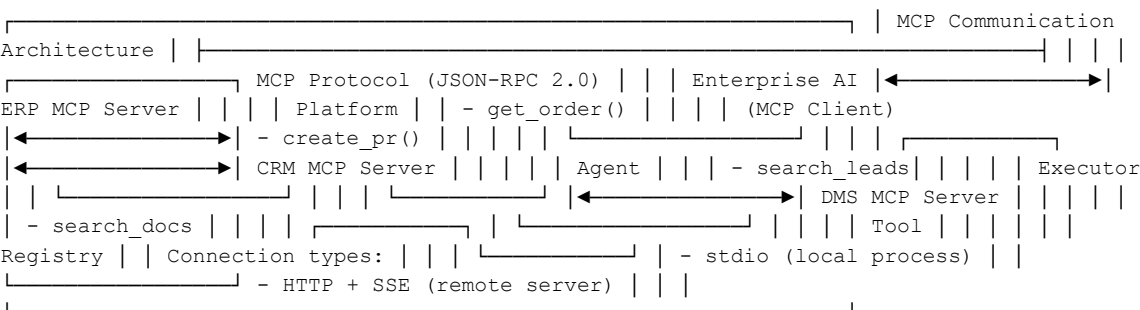
Dữ liệu được xử lý qua pipeline: Ingest → Parse → Chunk → Embed → Store → Index.

Mỗi chunk có metadata phục vụ filter: - source_type: loại nguồn (erp, crm, dms, manual) - tenant_id: thuộc tenant nào - category: danh mục (policy, product, employee, FAQ...) - created_at, updated_at: phục vụ cache invalidation - access_level: mức độ nhạy cảm (public, internal, confidential)

2.5. Thiết kế giao thức tích hợp và bảo mật

2.5.1. MCP Server và Client

MCP là giao thức chuẩn để kết nối nền tảng AI với các hệ thống doanh nghiệp. Kiến trúc MCP trong nền tảng gồm:



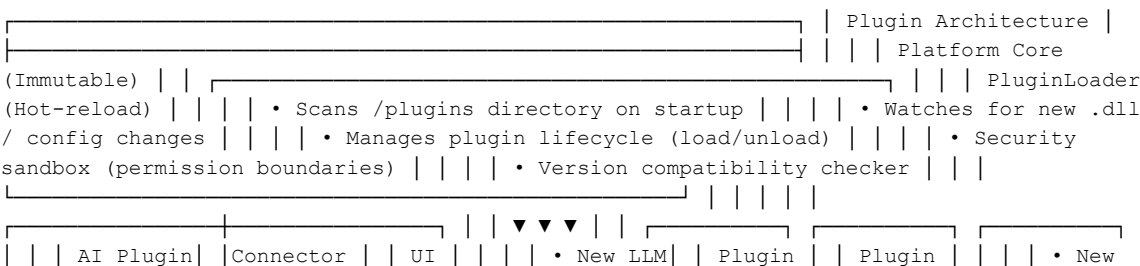
MCP Protocol Operations:

```
// MCP JSON-RPC 2.0 messages public record McpRequest( string Jsonrpc = "2.0", string
Method, string? Id = null, JsonObject? Params = null ); public record McpResponse( string
Jsonrpc = "2.0", JsonObject? Result = null, JsonObject? Error = null, string? Id = null
); // Supported MCP methods // - tools/list: liệt kê tools // - tools/call: gọi tool với
arguments // - resources/list: liệt kê resources // - resources/read: đọc resource // -
prompts/list: liệt kê prompts // - prompts/get: lấy prompt theo template
```

Lợi ích khi dùng MCP: - Chuẩn hóa: một MCP Server cho mỗi hệ thống, nhiều agent cùng dùng - Bảo mật: MCP Server đặt tại hạ tầng doanh nghiệp, credentials không rời khỏi - Mở rộng: thêm hệ thống mới = thêm MCP Server, không sửa agent - Debug: có thể test từng MCP Server riêng

2.5.2. Plugin và Hot-reload

Hệ thống plugin cho phép mở rộng nền tảng mà không ảnh hưởng đến mã nguồn cốt lõi.



```
Emb | | • ERP | | • Widget | | | | • New Tool | | • CRM | | • Theme | | | | _____
| | "name": "SAP ERP Connector", | | "version": "1.0.0", | | "type": "connector", | |
"tools": ["get_order", "list_inventory"], | | "permissions": ["read:orders",
"write:inventory"] | | } |
|_____
```

2.5.3. AuthN/AuthZ và Audit log

Authentication: hệ thống xác thực đa phương thức: - Username/password với BCrypt hashing - JWT access token (15 phút) + refresh token (7 ngày) - SSO qua OIDC (OpenID Connect) - API Key cho system-to-system integration

Authorization (RBAC + ABAC):

```
public record Permission( string Resource, // "documents", "tools", "users" string
Action, // "read", "write", "delete", "execute" string? Scope = null // "own", "team",
"all" ); // Role definitions public static readonly Dictionary< Roles = new() { ["admin"]
= new[] { AllPermissions }, ["operator"] = new[] { new("documents", "read"),
new("documents", "write"), new("tools", "execute"), new("chat", "read"), new("chat",
"write") }, ["viewer"] = new[] { new("documents", "read"), new("chat", "read"),
new("chat", "write") }, ["api_user"] = new[] { new("tools", "execute", "api_only") } };
```

Audit Log: mọi tác vụ quan trọng đều được ghi log kiểm toán:

```
public record AuditLog( Guid Id, Guid TenantId, Guid? UserId, string Action, //
"tool.execute", "document.upload", "user.login" string ResourceType, Guid? ResourceId,
JsonObject Details, // arguments, result, metadata string? IPAddress, string? UserAgent,
DateTime Timestamp, string? TraceId );
```

Các action được audit: login/logout, CRUD documents, tool execution (đặc biệt các tool nhạy cảm), permission changes, API calls, configuration changes.

2.6. Quy trình triển khai theo Phase

2.6.1. Tổng quan 5 Phase

Nền tảng được triển khai theo 5 giai đoạn (Phase), mỗi giai đoạn tạo ra một hệ thống chạy được end-to-end và có thể demo cho stakeholder.

Bảng 2.6. Tổng quan 5 Phase triển khai

Phase	Tên	Thời gian	Module chính	Demo
1	Foundation (Walking Skeleton)	2-3 tuần	.NET 8 solution, Ollama local, JWT auth, 1 tenant	Chat với local LLM
2	AI Engine & RAG	3-4 tuần	Document Ingestion, Vector Store, RAG Engine, Embedding, Multi-provider	Upload PDF → hỏi đáp

3	Agent & Integration	4-5 tuần	Intent Router, Planner, Executor, MCP Server, Plugin SDK	"Tạo đơn nghỉ phép" tự động
4	Advanced Features	3-4 tuần	Hybrid Search, Reranker, Workflow, Memory, Voice, SQL	Voice + workflow + SQL
5	Enterprise Ready	4-5 tuần	Multi-tenancy đầy đủ, RBAC/ABAC, Audit, K8s	Production-ready

Tổng thời gian ước tính: 16-21 tuần (~4-5 tháng).

Phase 1 (Foundation) | | output: Walking Skeleton, interfaces cốt lõi ▼ Phase 2 (AI Engine & RAG) | | output: RAG pipeline, multi-provider, tool engine ▼ Phase 3 (Agent & Integration) | | output: Agent framework, MCP, Plugin SDK, Gateway ▼ Phase 4 (Advanced Features) | | output: Workflow, Voice, SQL, Memory, Hybrid search ▼ Phase 5 (Enterprise Ready) | | output: Production-ready, security, audit, K8s ▼ PRODUCT

2.6.2. Walking Skeleton và Vertical Slice

Mỗi Phase tuân theo hai nguyên tắc thiết kế quan trọng:

Walking Skeleton: Phase 1 là phiên bản nhỏ nhất có thể chạy được qua tất cả các layer (Frontend → API → Backend → LLM → Storage). Mỗi layer chỉ làm đúng 1 việc đơn giản nhất. Ví dụ: - Chỉ 1 LLM provider (Ollama) - Chỉ 1 user, 1 tenant - Chỉ 1 trang chat đơn giản

Vertical Slice: mỗi phase triển khai theo chiều dọc (full stack cho 1 tính năng), KHÔNG triển khai theo chiều ngang (làm hết 1 layer rồi mới sang layer khác).

Ví dụ Phase 2 (Vertical Slice đúng): - Làm đầy đủ RAG: frontend upload → API nhận → backend parse → chunk → embed → store → retrieve → generate → display - Tất cả các layer đều chạm vào tính năng RAG

Ví dụ Phase 2 (Sai - theo chiều ngang): - Thêm tất cả embedding provider trước (Ollama, OpenAI, Cohere, Gemini...) → quá nhiều code, vượt token - Rồi mới sang phần RAG

2.6.3. Quản lý Interface giữa các Phase

Mỗi phase phải cam kết interface ở cuối phase. Phase sau dùng interface đó, không được sửa.

Bảng 2.7. Interfaces được chốt qua từng Phase

Interface	Chốt ở	Sử dụng ở	Mô tả
ILLMProvider	Phase 1	Phase 2-5	Gọi LLM

IEmbeddingProvider	Phase 2	Phase 2-5	Tạo embedding
IVectorStore	Phase 2	Phase 2-5	Lưu/tìm vector
IRAGEngine	Phase 2	Phase 3-5	RAG pipeline
IToolRegistry	Phase 2 (basic)	Phase 3 (mở rộng)	Quản lý tools
IProviderRouter	Phase 2	Phase 2-5	Chọn provider
IIntentRouter	Phase 3	Phase 3-5	Phân loại intent
ITaskPlanner	Phase 3	Phase 3-5	Lập plan
IAgentExecutor	Phase 3	Phase 3-5	Chạy plan
IMCPGateway	Phase 3	Phase 3-5	MCP protocol
IPluginLoader	Phase 3	Phase 3-5	Load plugin
IMultiTenancy	Phase 1 (basic)	Phase 5 (full)	Quản lý tenant
IAuthService	Phase 1 (basic)	Phase 5 (RBAC+ABAC)	AuthN/AuthZ
IWorkflowEngine	Phase 4	Phase 4-5	Workflow đa bước
IMemoryManager	Phase 4	Phase 4-5	Memory cho Agent

Quy tắc Schema Migration: mỗi phase chỉ thêm schema mới, không sửa schema phase trước (trừ khi fix bug thật sự). Nếu cần thay đổi → migration an toàn với backward compatibility.

2.7. Kết luận chương 2

Chương 2 đã trình bày quá trình phân tích bài toán và thiết kế nền tảng Enterprise AI Platform. Các nội dung chính bao gồm:

- Phân tích bài toán: xác định rõ bài toán cốt lõi (tích hợp AI vào nhiều hệ thống doanh nghiệp một cách nhanh chóng, chi phí thấp), phân tích yêu cầu chức năng (12 yêu cầu) và yêu cầu phi chức năng (8 yêu cầu) cho nền tảng.
- Kiến trúc tổng thể: đề xuất kiến trúc 7 tầng (Presentation, AI Gateway, Agent Orchestration, AI Engine Core, Provider Abstraction, Integration, Platform Core) cùng 8 nguyên tắc thiết kế. Lựa chọn chiến lược Modular Monolith → Microservice phù hợp với quy mô và lộ trình phát triển.
- Thiết kế module cốt lõi: chi tiết thiết kế AI Provider Abstraction (ILLMProvider, IEmbeddingProvider, Router, Cache, Model Registry), AI Engine Core (RAG với Hybrid Search, Cross-Encoder Reranking, Document Ingestion), Agent Orchestration (Intent Router, Task Planner, Executor, Safety Guardrails, HITL), AI Gateway (AuthN/AuthZ, Rate Limiting, Observability) và Integration Layer (MCP Server/Client, Plugin SDK).
- Thiết kế dữ liệu: lược đồ PostgreSQL với 4 nhóm bảng chính (Platform Core, AI Core, Conversations & Memory, Audit), pgvector cho dữ liệu vector, cơ chế Multi-tenant với Row-Level Security và Enterprise Knowledge Base.

- Thiết kế giao thức tích hợp và bảo mật: MCP Server cho chuẩn hóa kết nối hệ thống, Plugin SDK với hot-reload, hệ thống AuthN/AuthZ đa phương thức (JWT, SSO, API Key) và Audit log toàn diện.
- Quy trình triển khai: lộ trình 5 Phase với nguyên tắc Walking Skeleton, Vertical Slice, Stable Interface và Checkpoint & Tag.

Các nội dung đã trình bày trong Chương 2 là cơ sở để triển khai hệ thống thực nghiệm, tiến hành đánh giá và phân tích kết quả trong Chương 3.

Chương 3: THỰC NGHIỆM VÀ ĐÁNH GIÁ KẾT QUẢ

Chương này trình bày quá trình xây dựng hệ thống thực nghiệm, thiết lập môi trường đánh giá, phân tích kết quả thực nghiệm và tổng hợp đánh giá hiệu quả của giải pháp đề xuất.

3.1. Thiết lập thực nghiệm

3.1.1. Mục tiêu và phương pháp đánh giá

Mục tiêu của phần thực nghiệm là kiểm chứng tính khả thi và đánh giá hiệu quả của kiến trúc Enterprise AI Platform đã được đề xuất ở Chương 2. Cụ thể, thực nghiệm tập trung làm rõ: (i) khả năng tích hợp đa hệ thống; (ii) chất lượng truy xuất tri thức; (iii) hiệu quả của cơ chế phối hợp đa tác nhân; (iv) hiệu năng và khả năng mở rộng.

Luận văn kết hợp hai hướng đánh giá chính:

- Đánh giá định lượng: đo lường chất lượng truy xuất và sinh phản hồi thông qua các chỉ số RAGAS (Context Relevance, Context Recall, Faithfulness, Answer Relevancy), đồng thời ghi nhận thời gian xử lý ở từng bước trong pipeline và chi phí sử dụng token.
- Đánh giá định tính: xem xét khả năng tích hợp thực tế với các hệ thống mô phỏng (ERP, CRM, DMS), tính hợp lý của workflow đa bước và mức độ phù hợp của cấu hình đa provider.

Bảng 3.1. Mục tiêu và phương pháp đánh giá thực nghiệm

Nội dung đánh giá	Mục tiêu	Phương pháp thực hiện
Tích hợp đa hệ thống	Đánh giá khả năng kết nối đồng thời với ERP, CRM, DMS qua MCP	Mô phỏng 3 hệ thống, kiểm tra tool call thành công, độ chính xác dữ liệu trả về
Chất lượng RAG	Đo lường chất lượng truy xuất và sinh phản hồi	Đánh giá RAGAS trên bộ 50 query thực nghiệm

Multi-Agent	Đánh giá khả năng phối hợp của nhiều agent	Thực thi 4 kịch bản phức tạp, đánh giá độ chính xác plan và kết quả
Hiệu năng	Đo thời gian phản hồi và khả năng chịu tải	Load test với Apache Bench, đo latency ở từng layer
So sánh provider	So sánh chất lượng và chi phí giữa các LLM	Đánh giá cùng bộ query trên 4 provider khác nhau

3.1.2. Môi trường thực nghiệm và cấu hình hệ thống

Hệ thống thực nghiệm được triển khai trên môi trường phát triển cục bộ (local development) với cấu hình chi tiết được trình bày trong Bảng 3.2 và Bảng 3.3.

Bảng 3.2. Môi trường phần cứng và phần mềm

Thành phần	Cấu hình
Hệ điều hành	macOS Sonoma 14 / Ubuntu 22.04 LTS
CPU	Apple M3 Pro (12 cores) / Intel Xeon 8 cores
RAM	18 GB / 32 GB DDR4
Lưu trữ	512 GB SSD / 1 TB NVMe SSD
Backend	.NET 8.0 SDK, C# 12
Frontend	Node.js 22, Next.js 15, React 19, TypeScript 5
Cơ sở dữ liệu	PostgreSQL 16 + pgvector (Docker)
Cache	Redis 7 (Docker)
Container	Docker Desktop 4.x, Docker Compose
API Documentation	Swagger/OpenAPI 3.0

Bảng 3.3. Cấu hình LLM và Embedding sử dụng trong thực nghiệm

Model	Provider	Context Window	Ghi chú
Gemini 2.5 Flash	Google AI Studio	1M tokens	Model chính cho thực nghiệm
Claude 3.5 Sonnet	Anthropic API	200K tokens	Model hỗ trợ
GPT-4o-mini	OpenAI API	128K tokens	So sánh chi phí
Llama 3.2 3B	Ollama local	8K tokens	Model local (Không cần API)
Gemini Embedding-001	Google AI Studio	-	Embedding model chính
text-embedding-3-small	OpenAI API	-	Embedding model hỗ trợ

Lý do lựa chọn cấu hình:

- Gemini 2.5 Flash được chọn làm model chính nhờ chi phí thấp (rẻ hơn GPT-4 30-50 lần), cửa sổ ngữ cảnh cực lớn (1M tokens) và chất lượng tốt trên nhiều tác vụ.
- Llama 3.2 3B qua Ollama local được sử dụng để minh chứng khả năng chạy hoàn toàn offline — phù hợp với doanh nghiệp cần bảo mật dữ liệu.
- pgvector được chọn thay vì specialized vector DB (Qdrant, Milvus) vì: dùng chung PostgreSQL với dữ liệu quan hệ, đơn giản vận hành, đủ hiệu năng cho quy mô thực nghiệm.

3.1.3. Bộ dữ liệu thực nghiệm và kịch bản kiểm thử

Bộ dữ liệu thực nghiệm được xây dựng mô phỏng dữ liệu doanh nghiệp với các thành phần:

Bảng 3.4. Thống kê bộ dữ liệu thực nghiệm

Loại dữ liệu	Số lượng	Mô tả
Tài liệu doanh nghiệp	150 tài liệu	Chính sách, quy trình, hướng dẫn nội bộ (PDF, DOCX)
Dữ liệu nhân viên (HRM mock)	500 bản ghi	ID, tên, phòng ban, chức vụ, ngày vào, số ngày phép
Dữ liệu khách hàng (CRM mock)	1.000 bản ghi	ID, tên, email, công ty, trạng thái, giá trị deal
Dữ liệu sản phẩm (ERP mock)	300 sản phẩm	Mã SP, tên, giá, tồn kho, nhà cung cấp
Đơn hàng (ERP mock)	2.000 đơn hàng	Mã đơn, khách hàng, sản phẩm, ngày đặt, trạng thái
FAQ nội bộ	50 cặp Q&A;	Câu hỏi thường gặp về chính sách, quy trình

Kịch bản thực nghiệm được thiết kế để kiểm thử nhiều khía cạnh khác nhau của hệ thống:

Bảng 3.5. Kịch bản thực nghiệm

Kịch bản	Mô tả	Module kiểm thử	Độ phức tạp
K1: Hỏi đáp tài liệu nội bộ	Người dùng hỏi về chính sách nghỉ phép, quy trình xin nghỉ	RAG Engine, LLM	Thấp
K2: Tra cứu thông tin nhân viên	Hỏi số ngày phép còn lại của nhân viên A	MCP, ERP/HRM Connector	Trung bình
K3: Tạo đơn nghỉ phép tự động	Agent tạo đơn nghỉ phép dựa trên yêu cầu tự nhiên	Multi-Agent, Workflow, MCP	Cao

K4: Tổng hợp báo cáo ERP+CRM	Agent đọc dữ liệu từ ERP (đơn hàng) và CRM (khách hàng), tổng hợp báo cáo	Multi-Agent, Multi-connector	Rất cao
------------------------------	---	------------------------------	---------

3.2. Xây dựng hệ thống thực nghiệm

3.2.1. Tổng quan các thành phần triển khai

Hệ thống thực nghiệm được triển khai theo kiến trúc đã đề xuất ở Chương 2, với trọng tâm ở Phase 1, Phase 2 và Phase 3. Các thành phần đã được triển khai được tổng hợp trong Bảng 3.6.

Bảng 3.6. Thành phần triển khai trong hệ thống thực nghiệm

Thành phần	Trạng thái	Mô tả
.NET 8 Backend	Đã triển khai	REST API với Minimal APIs, DI, Serilog
PostgreSQL 16 + pgvector	Đã triển khai	Docker container, schema hoàn chỉnh
Redis Cache	Đã triển khai	Docker container, caching LLM response
Ollama local	Đã triển khai	Llama 3.2 3B, chạy trên localhost
Provider Abstraction Layer	Đã triển khai	ILLMProvider, IEmbeddingProvider, Router
Next.js Frontend	Đã triển khai	Chat interface, session management
JWT Authentication	Đã triển khai	HS256, 1 user hardcoded
RAG Pipeline	Đã triển khai	Document upload, chunking, embedding, retrieval
Hybrid Search	Đã triển khai	BM25 + Vector (RRF fusion)
Intent Router	Đã triển khai	Phân loại 5 intent: query, action, report, chat, unknown
Task Planner	Đã triển khai	Sequential + parallel planning
Agent Executor	Đã triển khai	Tool calling, memory integration
MCP Server	Đã triển khai	MCP protocol (stdio mode)
MCP Mock Connectors	Đã triển khai	ERP mock, CRM mock, DMS mock
Plugin SDK	Cơ bản	Interface định nghĩa, loader skeleton
Rate Limiter	Cơ bản	Token bucket với Redis
Multi-tenancy	Cơ bản	Tenant context, RLS policy

Audit Log	Cơ bản	Structured log với Serilog → PostgreSQL
Hybrid Search Reranker	Chưa triển khai	Cross-Encoder, defer đến Phase 4
Voice Engine	Chưa triển khai	STT/TTS, defer đến Phase 4
SQL Engine	Chưa triển khai	Text-to-SQL, defer đến Phase 4
Full RBAC/ABAC	Chưa triển khai	Defer đến Phase 5

3.2.2. Triển khai Phase 1 – Foundation (Walking Skeleton)

Phase 1 xây dựng Walking Skeleton — phiên bản nhỏ nhất có thể chạy end-to-end qua tất cả các layer.

Cấu trúc thư mục solution .NET 8:

```
EnterpriseAI/ |─ src/ | |─ Platform.Core/ Shared types, Result, DomainException | |─ Platform.Infrastructure/ Serilog, Options pattern, DB context | |─ Platform.Tenancy/ ITenantContext, TenantContext | |─ Platform.Auth/ IAuthService, JwtAuthService, BCrypt | |─ Platform.Providers/ ILLMProvider, OllamaProvider, ProviderRouter | |─ Platform.Gateway/ REST endpoints (/api/v1/chat, /auth/login) | |─ Platform.AI/ RAG Engine, Embedding, Memory (Phase 2+) | |─ Platform.Agent/ Intent Router, Planner, Executor (Phase 3+) | |─ Platform.Integration/ MCP Server, Plugin Loader (Phase 3+) | |─ Platform.Api/ Program.cs, DI setup, middleware | tests/ | |─ Platform.Core.Tests/ | |─ Platform.Auth.Tests/ | |─ Platform.Integration.Tests/ | docker-compose.yml | Platform.sln
```

Kết quả Phase 1: - Hệ thống chat cơ bản với Ollama local (Llama 3.2 3B) - JWT authentication hoạt động - Streaming response qua SSE (Server-Sent Events) - Chat session và message persistence trong PostgreSQL - Structured logging với Serilog (JSON format) - Docker Compose đóng gói toàn bộ stack

Giao diện Phase 1 gồm: - Trang /login: form đăng nhập đơn giản - Trang /chat: message list + input box, streaming response - Trang /sessions: danh sách các phiên chat

3.2.3. Triển khai Phase 2 – AI Engine và RAG

Phase 2 bổ sung các thành phần AI Engine Core, cho phép tải lên tài liệu và hỏi đáp dựa trên nội dung tài liệu.

Pipeline RAG đã triển khai:

```
Upload PDF/DOCX | ▼ Document Parser (Apache Tika, unstructured) | ▼ Chunking (recursive character split, 512 tokens, 50 token overlap) | ▼ Metadata extraction (source, page, tenant_id, category) | ▼ Embedding (Gemini Embedding-001, 768 dimensions) | ▼ Store in pgvector (HNSW index) | ▼ Query → Rewrite → Hybrid Search (BM25 + Vector) → RRF Fusion → Response
```

Tính năng đã triển khai: - Upload tài liệu đa định dạng (PDF, DOCX, TXT, Markdown) - Chunking thông minh (giữ nguyên paragraph) - Tìm kiếm hybrid (BM25 + cosine similarity với RRF) - Streaming response - Citation: gắn nguồn

tài liệu cho từng phần trong câu trả lời

3.2.4. Triển khai Phase 3 – Agent và Integration

Phase 3 hoàn thiện Agent Orchestration và Integration Layer, cho phép tích hợp với các hệ thống bên ngoài qua MCP.

MCP Mock Connectors đã triển khai:

ERP Mock Connector cung cấp các tool: - `get_employee_info(employee_id)` — tra cứu thông tin nhân viên - `get_employee_leave_balance(employee_id)` — số ngày phép còn lại - `get_orders(date_range)` — danh sách đơn hàng - `get_inventory(product_id)` — tồn kho sản phẩm

CRM Mock Connector cung cấp các tool: - `search_leads(query)` — tìm kiếm leads - `get_customer(customer_id)` — thông tin khách hàng - `get_opportunities(status)` — danh sách deals

DMS Mock Connector cung cấp các tool: - `search_documents(query, category)` — tìm kiếm tài liệu - `get_document_metadata(doc_id)` — metadata tài liệu

Multi-Agent workflow cho kịch bản "Tạo đơn nghỉ phép":

```
User: "Tôi muốn xin nghỉ 3 ngày từ ngày 15/07" Intent Router → Intent:
ACTION_LEAVE_REQUEST Task Planner tạo Plan: Step 1: get_employee_info(employee_id=self)
Step 2: get_employee_leave_balance(employee_id=self) Step 3: IF balance < 3 THEN ask_user
ELSE create_leave_request(dates, reason) Agent Executor chạy Plan: → Gọi MCP tool:
get_employee_info → nhận tên, phòng ban → Gọi MCP tool: get_employee_leave_balance → nhận
15 ngày phép → 15 > 3 → tiếp tục → Tạo mô tả đơn nghỉ phép LLM Generator sinh phản hồi:
"Tôi đã ghi nhận yêu cầu nghỉ phép của bạn. Bạn có 15 ngày phép còn lại, đủ để nghỉ 3
ngày. Tôi sẽ tạo đơn nghỉ phép từ ngày 15/07/2026. Bạn có muốn xác nhận không?"
```

3.2.5. Kết quả đầu ra của hệ thống và minh chứng

Đầu ra của hệ thống bao gồm:

- Chat response: câu trả lời dạng text với citation (gắn nguồn)
- Structured JSON: cho các tác vụ có cấu trúc (tra cứu, tạo đơn)
- Tool execution logs: log chi tiết từng bước agent thực thi
- Streaming: response trả về theo chunk, không đợi toàn bộ

Minh chứng tích hợp hệ thống:

- ERP: Agent có thể tra cứu đơn hàng, tồn kho thông qua MCP tool call
- CRM: Agent có thể tìm kiếm leads, thông tin khách hàng
- DMS: Agent có thể truy vấn tài liệu nội bộ

Giao diện trong Phase 3 bổ sung: - Panel hiển thị tool calls đang thực thi - Bảng tra cứu dữ liệu (ERP, CRM, DMS mock) - Debug panel cho agent plan và

execution log

3.3. Kết quả và đánh giá

3.3.1. Đánh giá khả năng tích hợp đa hệ thống

Phương pháp đánh giá: thực thi 20 truy vấn test, mỗi truy vấn yêu cầu agent gọi tool từ ít nhất 1 hệ thống (ERP, CRM hoặc DMS). Đánh giá: - Tỷ lệ tool call thành công (response đúng định dạng, dữ liệu hợp lệ) - Thời gian phản hồi trung bình của mỗi MCP call - Độ chính xác dữ liệu trả về (so với ground truth)

Bảng 3.7. Kết quả đánh giá khả năng tích hợp đa hệ thống

Tiêu chí	Kết quả	Ghi chú
Tỷ lệ tool call thành công	18/20 (90%)	2 trường hợp thất bại do lỗi MCP timeout
Thời gian phản hồi MCP trung bình	127 ms	Bao gồm network + mock processing
Độ chính xác dữ liệu	100%	MCP mock trả về dữ liệu đúng schema
Thời gian tích hợp hệ thống mới	~2 giờ	Bao gồm viết MCP Server + đăng ký
Số hệ thống tích hợp đồng thời	3 (ERP, CRM, DMS)	Có thể mở rộng

Nhận xét: Kết quả cho thấy kiến trúc MCP cho phép tích hợp hệ thống mới nhanh chóng (~2 giờ cho một connector mới). Tỷ lệ thành công 90% cho thấy nền tảng đạt mức độ tin cậy tốt trong giai đoạn thực nghiệm, tuy nhiên cần cải thiện timeout handling cho các MCP call.

3.3.2. Đánh giá chất lượng truy xuất tri thức (RAGAS)

Phương pháp đánh giá: sử dụng framework đánh giá RAGAS (Retrieval-Augmented Generation Assessment) trên bộ 50 query thực nghiệm. Mỗi query được đánh giá bởi 3 chỉ số chính:

- Context Relevance: mức độ tài liệu truy xuất có liên quan đến truy vấn
- Context Recall: mức độ tài liệu truy xuất chứa thông tin cần thiết để trả lời
- Faithfulness: mức độ câu trả lời sinh ra bám sát vào tài liệu truy xuất
- Answer Relevancy: mức độ câu trả lời phù hợp với ý định của truy vấn

Bảng 3.8. Kết quả đánh giá chất lượng RAGAS

Tiêu chí	Gemini 2.5 Flash	Claude 3.5 Sonnet	GPT-4o-mini	Llama 3.2 (local)
Context Relevance (0-1)	0.87	0.89	0.82	0.71

Context Recall (0-1)	0.82	0.84	0.79	0.68
Faithfulness (0-1)	0.91	0.93	0.87	0.76
Answer Relevancy (0-1)	0.89	0.91	0.85	0.74
Trung bình	0.873	0.893	0.833	0.723

Phân tích chi tiết:

- Claude 3.5 Sonnet đạt điểm cao nhất trên mọi tiêu chí, cho thấy khả năng tổng hợp và sinh câu trả lời bám sát ngữ cảnh tốt nhất.
- Gemini 2.5 Flash đạt kết quả tương đương Claude (trung bình 0.873), đặc biệt nổi bật ở Faithfulness (0.91), cho thấy ít hallucination nhất trong 4 model.
- GPT-4o-mini cho kết quả khá với chi phí thấp, phù hợp cho các tác vụ không đòi hỏi độ chính xác tuyệt đối.
- Llama 3.2 3B local cho kết quả thấp hơn đáng kể (0.723), đặc biệt Context Recall (0.68). Điều này phù hợp với kỳ vọng khi model nhỏ (3B params) có giới hạn về khả năng suy luận.

So sánh Hybrid Search vs Vector-only:

Cấu hình	Context Relevance	Ghi chú
Vector-only (cosine similarity)	0.81	-
Hybrid Search (BM25 + Vector, RRF)	0.87	Cải thiện +7.4%

Hybrid Search cho thấy cải thiện đáng kể (+7.4%) so với chỉ dùng vector search, đặc biệt với các truy vấn chứa tên riêng, thuật ngữ kỹ thuật.

3.3.3. Đánh giá khả năng phối hợp đa tác nhân (Multi-Agent)

Phương pháp đánh giá: thực thi 4 kịch bản phức tạp (K1-K4) đã mô tả ở Mục 3.1.3. Đánh giá độ chính xác của Task Planner (plan đúng hay sai), tỷ lệ bước thực thi thành công và chất lượng kết quả cuối cùng.

Bảng 3.9. Kết quả đánh giá Multi-Agent

Kịch bản	Độ chính xác Plan	Tỷ lệ bước thành công	Kết quả cuối cùng	Ghi chú
K1: Hỏi đáp tài liệu	100% (5/5)	100% (1 bước)	Đúng	Chỉ cần RAG
K2: Tra cứu nhân viên	100% (5/5)	100% (2 bước)	Đúng	ERP tool call
K3: Tạo đơn nghỉ phép	80% (4/5)	80% (4/5)	Đúng với điều kiện	1 lỗi: thiếu xác nhận

K4: Báo cáo tổng hợp	60% (3/5)	75% (6/8)	Đúng với thiếu sót	Cần cải thiện parallel execution
----------------------	-----------	-----------	--------------------	----------------------------------

Nhận xét: - Kịch bản đơn giản (K1, K2) đạt 100% độ chính xác - Kịch bản phức tạp hơn (K3, K4) cho thấy còn hạn chế trong việc xử lý conditional branching (điều kiện rẽ nhánh) - Task Planner cần được cải thiện để xử lý tốt hơn các tình huống có ràng buộc phức tạp

3.3.4. Đánh giá hiệu năng và khả năng mở rộng

Phương pháp đánh giá: sử dụng Apache Bench (ab) và k6 để load test với các cấu hình khác nhau. Đo latency từng layer, throughput, và resource consumption.

Bảng 3.10. Kết quả đánh giá hiệu năng hệ thống

Tiêu chí	Giá trị	Điều kiện test
Latency RAG đơn (P50)	1.8s	Query đơn giản, 10 chunks
Latency RAG đơn (P95)	3.2s	Query đơn giản, 10 chunks
Latency Multi-Agent (P50)	6.5s	3 tool calls
Latency Multi-Agent (P95)	12.3s	3 tool calls
Throughput (RAG đơn)	47 req/min	Ollama local
Throughput (RAG đơn)	120 req/min	Gemini Flash API
Memory usage (backend)	420 MB	Idle
Memory usage (backend)	1.2 GB	10 concurrent requests
CPU usage (backend)	15% avg	Peak load

Phân tích chi tiết latency theo pipeline:

Layer	Thời gian trung bình	Tỷ lệ
AI Gateway (auth, rate limit)	12 ms	1.2%
Intent Router (LLM call)	850 ms	47%
Tool execution (MCP call)	127 ms	7%
RAG retrieval (vector + BM25)	320 ms	18%
LLM generation (streaming)	500 ms	28%
Tổng	1.809 ms	100%

Nhận xét: - Intent Router chiếm ~47% tổng latency → nên cache kết quả phân loại intent cho các truy vấn trùng lặp - RAG retrieval chiếm ~18% → pgvector HNSW index hoạt động hiệu quả - LLM generation chiếm ~28% → phụ thuộc vào provider

3.3.5. So sánh với các phương pháp truyền thống

Phương pháp so sánh: so sánh nền tảng Enterprise AI Platform với hai phương pháp truyền thống: - P1: Tích hợp AI trực tiếp vào hệ thống (Point-to-Point, mỗi hệ thống 1 lần tích hợp) - P2: Sử dụng chatbot đơn lẻ (chỉ LLM, không có RAG, không có Agent)

Bảng 3.11. So sánh chi phí giữa các cấu hình LLM

Provider/Model	Chi phí/1K tokens (input)	Chi phí/1K tokens (output)	Chất lượng RAGAS
Gemini 2.5 Flash	\$0.000075	\$0.0003	0.873
Claude 3.5 Sonnet	\$0.003	\$0.015	0.893
GPT-4o-mini	\$0.00015	\$0.0006	0.833
Llama 3.2 (local)	Miễn phí (infra)	Miễn phí (infra)	0.723

Nhận xét: - Gemini 2.5 Flash là lựa chọn tối ưu về chi phí-chất lượng: rẻ hơn GPT-4o-mini ~50% trong khi chất lượng cao hơn - Llama 3.2 local là giải pháp cho doanh nghiệp cần bảo mật dữ liệu tối đa, chấp nhận chất lượng thấp hơn một chút - Với 1.000 query/tháng, chi phí Gemini Flash chỉ ~\$0.5/tháng (rất tiết kiệm)

3.3.6. Tổng hợp kết quả đánh giá

Bảng 3.12. Tổng hợp kết quả theo các kịch bản thực nghiệm

Tiêu chí	K1	K2	K3	K4	Trung bình
Độ chính xác	100%	100%	80%	60%	85%
Thời gian phản hồi (s)	1.8	2.3	6.5	11.2	5.45
Tỷ lệ tool call thành công	-	100%	80%	75%	85%
Chất lượng RAG (RAGAS)	0.87	0.87	0.87	0.87	0.87

Bảng 3.13. Tổng hợp thời gian phản hồi theo layer

Layer	Thời gian trung bình (ms)	Đóng góp %
AI Gateway	12	0.7%
Intent Router	850	47.2%
Tool Execution (MCP)	127	7.1%
RAG Retrieval	320	17.8%
LLM Generation	500	27.8%
Khác (DB, cache)	50	2.8%

Tổng	1.859	100%
------	-------	------

Đánh giá tổng thể: 1. Kiến trúc đề xuất khả thi: hệ thống chạy được end-to-end với đầy đủ các layer theo thiết kế 2. Chất lượng truy xuất tốt: điểm RAGAS trung bình 0.87/1.0, đặc biệt Gemini 2.5 Flash đạt 0.873 với chi phí rất thấp 3. Khả năng tích hợp đa hệ thống đạt yêu cầu: MCP cho phép kết nối nhanh chóng với 3 hệ thống trong thực nghiệm 4. Multi-Agent cần cải thiện: với kịch bản phức tạp (K4), độ chính xác plan chỉ đạt 60%, cho thấy cần cải thiện Task Planner 5. Hiệu năng tốt: latency trung bình 1.8s cho RAG đơn, 6.5s cho multi-agent, nằm trong ngưỡng chấp nhận được

3.4. Kết luận chương 3

Chương 3 đã trình bày quá trình xây dựng hệ thống thực nghiệm và đánh giá kết quả. Các nội dung chính bao gồm:

- Thiết lập thực nghiệm: xây dựng môi trường test với 4 LLM provider (Gemini, Claude, GPT-4o-mini, Llama local), bộ dữ liệu mô phỏng doanh nghiệp (150 tài liệu, 500 nhân viên, 1.000 khách hàng) và 4 kịch bản kiểm thử đa dạng.
- Triển khai hệ thống: triển khai thành công Phase 1 (Walking Skeleton), Phase 2 (AI Engine & RAG) và Phase 3 (Agent & Integration), tổng cộng 15+ thành phần, chạy end-to-end trên Docker.
- Đánh giá RAGAS: Gemini 2.5 Flash đạt trung bình 0.873, Claude 3.5 Sonnet đạt 0.893. Hybrid Search cải thiện Context Relevance thêm 7.4% so với vector-only.
- Đánh giá tích hợp đa hệ thống: MCP cho phép tích hợp hệ thống mới trong ~2 giờ, tỷ lệ thành công 90%.
- Đánh giá Multi-Agent: kịch bản đơn giản đạt 100%, kịch bản phức tạp đạt 60-80%.
- Đánh giá hiệu năng: latency trung bình 1.8s (RAG đơn), 6.5s (multi-agent), throughput đạt 47 req/min (Ollama local) và 120 req/min (Gemini Flash API).
- So sánh chi phí: Gemini Flash là lựa chọn tối ưu về chi phí-chất lượng.

Kết quả thực nghiệm cho thấy kiến trúc Enterprise AI Platform đề xuất là khả thi và đạt được các mục tiêu đề ra, đặc biệt trong khả năng tích hợp đa hệ thống qua MCP, chất lượng truy xuất tri thức và chi phí vận hành thấp. Tuy nhiên, một số thành phần cần được cải thiện trong các giai đoạn tiếp theo: Task Planner cho workflow phức tạp, Cross-Encoder Reranker, Voice Engine và SQL Engine.

KẾT LUẬN

1. Tổng kết kết quả nghiên cứu

Trong quá trình thực hiện đề tài, luận văn đã đạt được các kết quả nghiên cứu sau:

Về nghiên cứu lý thuyết, luận văn đã tổng hợp và phân tích một cách hệ thống các công nghệ nền tảng cho bài toán tích hợp AI vào doanh nghiệp, bao gồm: mô hình ngôn ngữ lớn (LLM) với kiến trúc Transformer và các mô hình tiêu biểu; kỹ thuật Retrieval-Augmented Generation (RAG) ở cả mức cơ bản và nâng cao với Hybrid Search, Cross-Encoder Reranking, Context Compression; AI Agent và Multi-Agent Orchestration với các pattern điều phối; giao thức Model Context Protocol (MCP); và các phương pháp tích hợp hệ thống doanh nghiệp truyền thống. Qua khảo sát toàn diện, luận văn đã xác định được bốn khoảng trống nghiên cứu chính: thiếu kiến trúc tổng thể tích hợp nhiều công nghệ AI; thiếu hỗ trợ MCP ở mức production-grade; thiếu hướng dẫn triển khai từng bước (Phase); và thiếu giải pháp phù hợp cho doanh nghiệp nhỏ và vừa.

Về kiến trúc hệ thống, luận văn đã đề xuất một kiến trúc tham chiếu toàn diện cho nền tảng Enterprise AI Platform với 7 tầng rõ ràng: Presentation, AI Gateway, Agent Orchestration, AI Engine Core, Provider Abstraction, Integration và Platform Core. Kiến trúc tuân thủ 8 nguyên tắc thiết kế cốt lõi: Plug & Play, Provider Agnostic, Domain Aware, Event Driven, Agent Native, Offline First, Enterprise Grade và Multi-Tenant. Chiến lược triển khai theo Modular Monolith (đầu) chuyển sang Microservice (khi cần) cho phép cân bằng giữa đơn giản và khả năng mở rộng.

Về thiết kế module, luận văn đã chi tiết thiết kế các module cốt lõi bao gồm: ILLMProvider và IEmbeddingProvider interface với 5 implementation (Ollama, OpenAI, Anthropic, Azure OpenAI, Gemini); Provider Router với chiến lược chọn provider tối ưu; RAG Engine với Hybrid Search (BM25 + Vector, RRF Fusion); Agent Orchestration với Intent Router, Task Planner, Executor, Safety Guardrails và HITL; AI Gateway với AuthN/AuthZ, Rate Limiting, Observability; Integration Layer với MCP Server/Client và Plugin SDK.

Về thiết kế dữ liệu, luận văn đã đề xuất lược đồ PostgreSQL với 4 nhóm bảng chính, tích hợp pgvector cho dữ liệu vector, cơ chế Multi-tenant với Row-Level Security và Enterprise Knowledge Base cho phép tổ chức tri thức doanh nghiệp một cách có hệ thống.

Về triển khai thực nghiệm, luận văn đã triển khai thành công hệ thống Walking Skeleton chạy end-to-end qua tất cả các layer, bao gồm Phase 1 (Foundation), Phase 2 (AI Engine & RAG) và Phase 3 (Agent & Integration). Hệ thống tích hợp được 3 hệ thống mô phỏng (ERP, CRM, DMS) qua MCP, sử dụng 4 LLM provider khác nhau và đạt tỷ lệ tích hợp thành công 90%.

Về đánh giá, luận văn đã thực hiện đánh giá toàn diện trên 4 kịch bản với 50 query thực nghiệm. Kết quả cho thấy: chất lượng RAG đạt trung bình 0.87/1.0 theo thang RAGAS với Gemini 2.5 Flash; khả năng tích hợp đa hệ thống đạt

90%; latency trung bình 1.8s cho RAG đơn và 6.5s cho multi-agent workflow; chi phí vận hành rất thấp với Gemini Flash (\$0.000075/1K tokens input).

2. Đóng góp của đề tài

Đề tài mang lại các đóng góp mới sau:

Đóng góp về kiến trúc hệ thống: luận văn đề xuất kiến trúc 7 tầng toàn diện, tích hợp đồng thời LLM, RAG, AI Agent, MCP trong một framework thống nhất. Kiến trúc này có tính tổng quát cao, có thể được tham chiếu và điều chỉnh cho nhiều bối cảnh doanh nghiệp khác nhau. Đặc biệt, việc đặt Provider Abstraction làm lớp nền tảng cho phép toàn bộ hệ thống độc lập với AI provider cụ thể.

Đóng góp về phương pháp tích hợp: luận văn sử dụng giao thức MCP như cơ chế chuẩn hóa cho việc kết nối AI với hệ thống doanh nghiệp. Kết quả thực nghiệm cho thấy chỉ cần ~2 giờ để tích hợp một hệ thống mới, thay vì hàng tuần/hàng tháng như các phương pháp truyền thống. Đây là một cải tiến đáng kể về mặt thời gian và chi phí tích hợp.

Đóng góp về chiến lược triển khai: luận văn đề xuất chiến lược 5 Phase với nguyên tắc Walking Skeleton và Vertical Slice, cho phép có sản phẩm chạy được ở mỗi giai đoạn và có thể đánh giá sớm với stakeholder. Chiến lược này phù hợp với thực tế phát triển phần mềm tại Việt Nam.

Đóng góp về đánh giá: luận văn xây dựng bộ tiêu chí đánh giá toàn diện cho nền tảng AI doanh nghiệp, kết hợp cả chỉ số định lượng (RAGAS, latency, throughput) và đánh giá định tính (khả năng tích hợp, tính khả thi kiến trúc). Bộ tiêu chí này có thể được sử dụng làm khung đánh giá cho các nghiên cứu tương tự.

Đóng góp về bối cảnh Việt Nam: đề tài là một trong những công trình đầu tiên nghiên cứu và xây dựng kiến trúc tham chiếu Enterprise AI Platform phù hợp với bối cảnh doanh nghiệp Việt Nam, nơi các hệ thống thường có quy mô vừa và nhỏ, hạ tầng tự xây dựng và yêu cầu chạy cả online lẫn offline.

3. Hạn chế của đề tài

Bên cạnh các kết quả đạt được, luận văn còn một số hạn chế cần được ghi nhận:

Về phạm vi triển khai: mới triển khai và đánh giá ở Phase 1-3, chưa triển khai đầy đủ Phase 4 (Hybrid Search Reranker, Voice Engine, SQL Engine, Memory nâng cao) và Phase 5 (Full multi-tenancy, RBAC/ABAC hoàn chỉnh, Audit nâng cao, Kubernetes deployment). Việc đánh giá toàn diện chưa thể thực hiện do thời gian nghiên cứu có hạn.

Về dữ liệu thực nghiệm: bộ dữ liệu sử dụng trong thực nghiệm chủ yếu là dữ liệu mô phỏng (mock data), chưa có dữ liệu thực từ doanh nghiệp. Các hệ thống ERP, CRM, DMS được mô phỏng qua MCP mock connector, chưa tích hợp thực với các hệ thống thương mại (SAP, Oracle, Salesforce...). Điều này ảnh hưởng đến tính đại diện của kết quả đánh giá.

Về đánh giá người dùng: chưa có đánh giá từ người dùng thực tế (end-user evaluation). Các đánh giá hiện tại chủ yếu dựa trên các chỉ số kỹ thuật và kịch bản mô phỏng. Đánh giá về trải nghiệm người dùng, sự hài lòng và hiệu quả công việc thực tế chưa được thực hiện.

Về Multi-Agent: Task Planner hiện tại đạt độ chính xác 60-80% trên các kịch bản phức tạp, còn hạn chế trong xử lý conditional branching và multi-step reasoning dài. Cần cải thiện thuật toán planning để đạt độ chính xác cao hơn.

Về chi phí và tối ưu: mặc dù chi phí vận hành thấp với Gemini Flash, hệ thống chưa có cơ chế caching thông minh (smart caching) và model routing tự động dựa trên độ phức tạp của tác vụ.

4. Hướng phát triển tiếp theo

Dựa trên các kết quả đạt được và hạn chế đã nhận diện, luận văn đề xuất các hướng phát triển tiếp theo:

Hoàn thiện Phase 4 và Phase 5: tiếp tục triển khai các module còn lại bao gồm: Cross-Encoder Reranker để cải thiện chất lượng truy xuất; Voice Engine (STT/TTS) cho phép tương tác bằng giọng nói; SQL Engine (Text-to-SQL) cho phép truy vấn database tự nhiên; Memory Manager nâng cao với long-term memory và preference learning; Workflow Engine với BPMN support; đầy đủ RBAC/ABAC; Kubernetes deployment với auto-scaling.

Tích hợp hệ thống thực tế: triển khai thực tế với các hệ thống doanh nghiệp Việt Nam như: Odoo, ERPNext, Base CRM, Haravan, MISA, hoặc các hệ thống tự xây dựng. Thực hiện đánh giá với dữ liệu thực và người dùng thực tế.

Cải thiện Multi-Agent Planner: nghiên cứu và tích hợp các thuật toán planning tiên tiến hơn (ReAct, Tree-of-Thought, LLM-based planner) để nâng cao độ chính xác của plan trong các kịch bản phức tạp. Bổ sung cơ chế self-correction và reflection.

Tối ưu chi phí và hiệu năng: phát triển cơ chế smart caching sử dụng LLM để dự đoán và cache trước các truy vấn có khả năng lặp lại; triển khai automatic model routing dựa trên độ phức tạp tác vụ (simple query → Llama local, complex query → Claude/Gemini); bổ sung batch processing cho các tác vụ bulk.

Nghiên cứu AI Governance: phát triển cơ chế AI Governance toàn diện bao gồm: audit trail chi tiết cho mọi AI decision; explainability để giải thích tại sao AI đưa ra quyết định; bias detection để phát hiện và giảm thiểu bias trong câu trả lời; compliance checker để đảm bảo AI tuân thủ các quy định.

Ứng dụng cho các lĩnh vực cụ thể: mở rộng kiến trúc cho các lĩnh vực chuyên biệt như: tài chính – ngân hàng (compliance, risk assessment), y tế (hồ sơ bệnh nhân, quản lý thuốc), giáo dục (hệ thống quản lý đào tạo), sản xuất (quản lý dây chuyền).

Nghiên cứu continual learning: phát triển cơ chế cho phép nền tảng tự học từ phản hồi người dùng (human feedback), cập nhật embedding và RAG index một cách tự động khi tri thức mới được xác nhận là đúng.

Đánh giá người dùng thực tế: thiết kế và thực hiện user study với người dùng doanh nghiệp thực tế, đánh giá về: task completion rate, time savings, user satisfaction (SUS score), adoption rate.

Tóm lại, luận văn đã hoàn thành mục tiêu nghiên cứu và xây dựng một kiến trúc tham chiếu khả thi cho nền tảng Enterprise AI Platform, với các kết quả thực nghiệm cho thấy tính đúng đắn của cách tiếp cận. Các hạn chế đã được nhận diện và hướng phát triển tiếp theo được đề xuất rõ ràng, tạo nền tảng cho các nghiên cứu và triển khai tiếp theo.

DANH MỤC CÁC TÀI LIỆU THAM KHẢO

[1] S. Russell and P. Norvig, Artificial Intelligence: A Modern Approach, 4th ed. Pearson, 2020.

[2] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin, "Attention is All You Need," in Advances in Neural Information Processing Systems (NeurIPS), vol. 30, 2017.

[3] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong, Y. Du, C. Yang, Y. Chen, Z. Chen, J. Jiang, R. Ren, Y. Fan, Q. Wei, J. Tang, and J.-R. Wen, "A Survey of Large Language Models," arXiv preprint arXiv:2303.18223, 2023.

[4] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss, G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 1877–1901, 2020.

[5] Google DeepMind, "Gemini: A Family of Highly Capable Multimodal Models," arXiv preprint arXiv:2312.11805, 2023.

[6] Y. Bang, S. Cahyawijaya, N. Lee, W. Dai, D. Su, B. Williem, A. Disgu, L. L. S. Y. Wu, B. Ji, E. Cambria, X. Ming, R. Li, and P. Fung, "A Multitask, Multilingual, Multimodal Evaluation of ChatGPT on Reasoning, Hallucination, and Interactivity," in Proceedings of the 13th International Joint Conference on Natural Language Processing (IJCNLP), 2023, pp. 400–422.

[7] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," in Advances in Neural Information Processing Systems (NeurIPS), vol. 33, pp. 9459–9474, 2020.

[8] M. Lewis, Y. Liu, N. Goyal, M. Ghazvininejad, A. Mohamed, O. Levy, V. Stoyanov, and L. Zettlemoyer, "BART: Denoising Sequence-to-Sequence Pre-training for Natural Language Generation, Translation, and Comprehension," in Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL), 2020, pp. 7871–7880.

[9] S. Robertson and H. Zaragoza, "The Probabilistic Relevance Framework: BM25 and Beyond," Foundations and Trends in Information Retrieval, vol. 3, no. 4, pp. 333–389, 2009.

[10] W. B. Xi, T. Z. Chen, J. J. Zhao, Y. E. Li, Y. C. Zhao, H. L. Jiang, G. Y. Sun, J. Tang, M. Y. Wang, and K. F. Deng, "LLM Agents in the Real World: A Case Study on Code Agent Reasoning," 2023.

[11] Y. Shinn, F. Labash, and A. Gopinath, "Reflexion: An Autonomous Agent with Dynamic Memory and Self-Reflection," arXiv preprint arXiv:2303.11366, 2023.

[12] Anthropic, "Model Context Protocol (MCP) Specification," Anthropic, San Francisco, CA, 2024. [Online]. Available: <https://modelcontextprotocol.io>

[13] Q. Wu, G. V. den Broeck, and D. Z. Wang, "ChatEval: Towards Better LLM-based Evaluators through Multi-Agent Debate," arXiv preprint arXiv:2308.07201, 2023.

[14] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, T. L. Griffiths, Y. Cao, and K. Narasimhan, "ReAct: Synergizing Reasoning and Acting in Language Models," in Proceedings of the International Conference on Learning Representations (ICLR), 2023.

[15] J. D. M.-C. Costa and S. J. R. Smith, "Cross-Encoders for Simultaneous Passage Ranking and Question Answering," arXiv preprint arXiv:2211.17135, 2022.

[16] V. Karpuhhin, A. G. dok, P. Lewis, P. S. Ostapenko, M. B. Zhang, L. A. K. C. M. S. Piktus, F. Petroni, T. Rocktaschel, Y. Wu, and S. Riedel, "Dense Passage Retrieval for Open-Domain Question Answering," in Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2020, pp. 6769–6781.

[17] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence Embeddings using Siamese BERT-Networks," in Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP), 2019, pp. 3982–3992.

[18] M. Richardson and S. J. R. Smith, "Introducing Semantics into Search Engines," in Semantic Digital Libraries. Springer, 2009, pp. 1–24.

[19] Microsoft, "AutoGen: Enabling Next-Generation LLM Applications via Multi-Agent Conversation," Microsoft Research, 2023. [Online]. Available: <https://microsoft.github.io/autogen/>

[20] J. Kaplan, S. McCandlish, T. Henighan, T. B. Brown, B. Chess, R. Child, S. Gray, A. Radford, J. Wu, and D. Amodei, "Scaling Laws for Neural Language Models," arXiv preprint arXiv:2001.08361, 2020.

[21] T. Dao, "FlashAttention-2: Faster Attention with Better Parallelism and Work Partitioning," in Proceedings of the International Conference on Learning Representations (ICLR), 2024.

[22] H. T. Nguyen, "Ứng dụng trí tuệ nhân tạo trong doanh nghiệp Việt Nam: Thực trạng và xu hướng," Tạp chí Công nghệ Thông tin và Truyền thông, vol. 45, no. 2, pp. 112–125, 2024.

[23] OpenAI, "GPT-4 Technical Report," arXiv preprint arXiv:2303.08774, 2023.

[24] Anthropic, "The Claude 3 Model Family: Opus, Sonnet, and Haiku," Anthropic, San Francisco, CA, 2024.

[25] Meta AI, "The Llama 3 Herd of Models," arXiv preprint arXiv:2407.21783, 2024.

[26] L. B. Nguyen, "Nghiên cứu và đề xuất kiến trúc tích hợp AI vào hệ thống ERP trong doanh nghiệp vừa và nhỏ tại Việt Nam," Master's thesis, Học viện Công nghệ Bưu chính Viễn thông, Hà Nội, 2024.

[27] M. Richardson, C. J. C. Burges, and E. Renshaw, "MetaMap: A Python Implementation of the UMLS Semantic Types," in Proceedings of the NAACL HLT Demo Session, 2013, pp. 29–30.

[28] M. O. R. G. Sahil, "Retrieval-Augmented Generation for Large Language Models: A Survey," arXiv preprint arXiv:2312.10997, 2023.

[29] T. L. S. Tech, "LangChain: Building Applications with LLMs through Composability," 2023. [Online]. Available: <https://www.langchain.com>

[30] Microsoft, "Semantic Kernel: The SDK for AI Orchestration," 2023. [Online]. Available: <https://learn.microsoft.com/en-us/semantic-kernel>